Raport de securitate al aplicatiei AuthX

Link GitHub: https://github.com/ArianaRobu/authx-break-login

Link video: https://drive.google.com/file/d/1nE-Jh_CfKGG0VfgW8UqNsxhqZXMWCJs3/view?usp=drive_link

Cuprins

1. Introducere
2. Setup mediu
3. Implementare MVP
4. Prezentare Vulnerabilități
5. Demonstrare atac (PoC)
6. Analiză Impact
7. Implementare Fix (v2 securizat)
8. Re-test – Dovadă că atacurile nu mai funcționează
9. Audit & Logging
10. Concluzii

1. Introducere

Descrierea aplicatiei

AuthX reprezinta o platforma digitala de tip Enterprise, conceputa special pentru ecosistemul intern al organizatiei. Obiectivul central al aplicatiei este securizarea si centralizarea managementului identitatilor, oferind personalului autorizat un punct unic de acces catre resursele critice si sistemul intern de suport tehnic.

Aplicatia implementeaza un flux complet de autentificare si autorizare, incluzand module de:

- Inregistrare si gestionare utilizatori.

- Autentificare securizata si management de sesiuni.
- Sistem de tichete pentru demonstrarea controlului accesului.
- Jurnale de audit pentru trasabilitatea actiunilor.

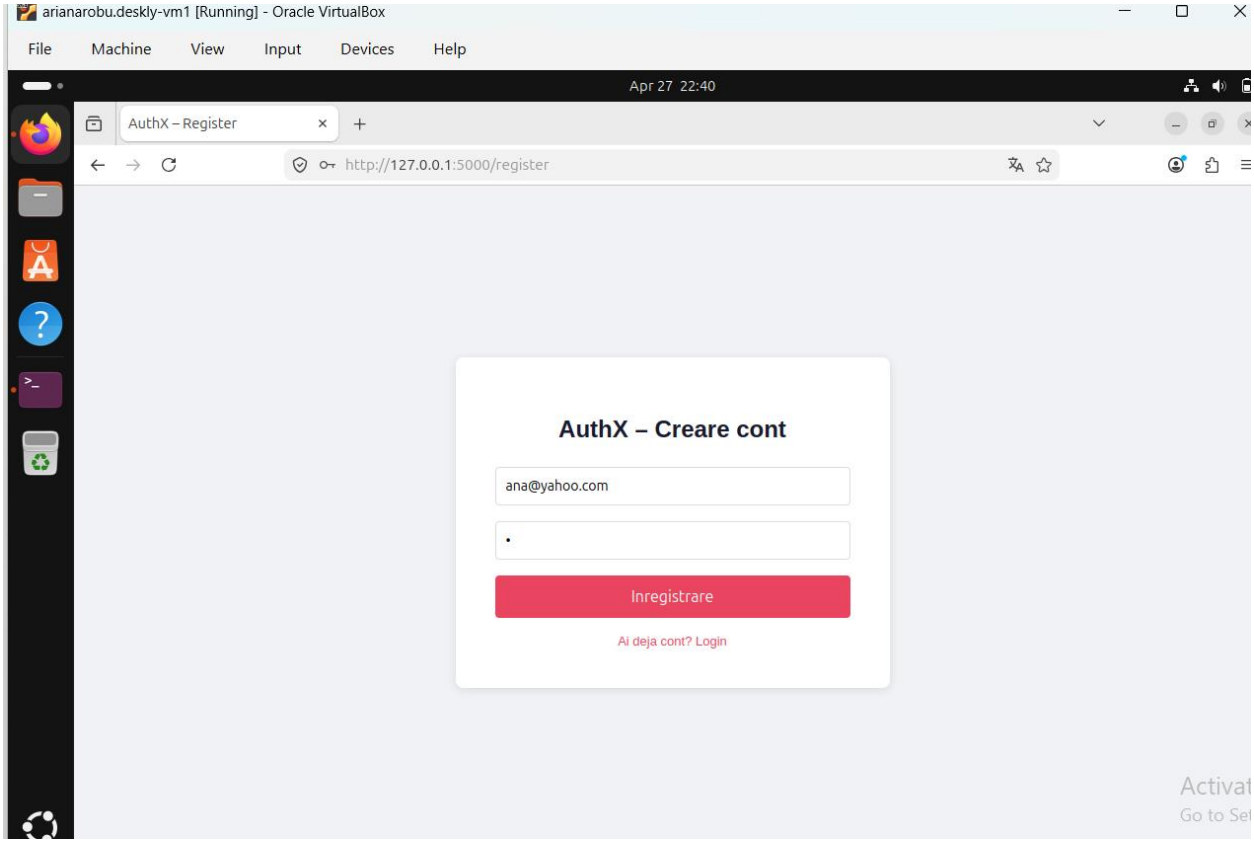
Scopul proiectului

Obiectivul principal este intelegerea practica a atacurilor asupra sistemelor de autentificare si implementarea masurilor de remediere. Proiectul urmeaza metodologia Secure SDLC (Build, Hack, Fix, Retest). In prima faza a fost construita o versiune functionala, dar vulnerabila intentionat (v1). Ulterior, au fost identificate si demonstrate vulnerabilitatile prin tehnici de ethical hacking, urmate de implementarea unei versiuni securizate (v2) care rezista atacurilor demonstrate.

Arhitectura aplicatiei

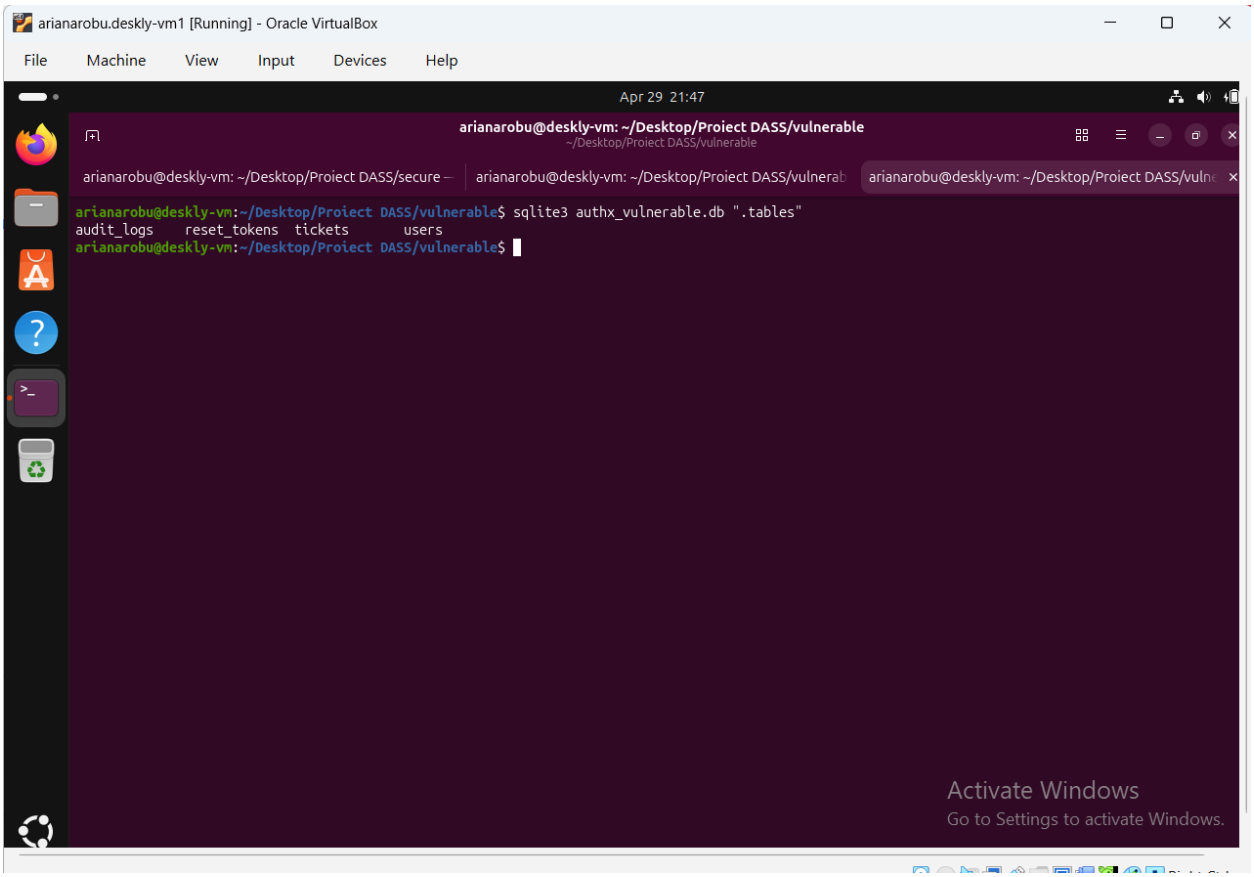
Aplicatia este structurata pe trei niveluri si ruleaza intr-un mediu izolat pe o masina virtuala:

Frontend: Interfata web bazata pe HTML si CSS servita prin motorul de sabloane Jinja2 din Flask.



Backend: API Flask in Python care gestioneaza logica de autentificare si regulile de securitate.

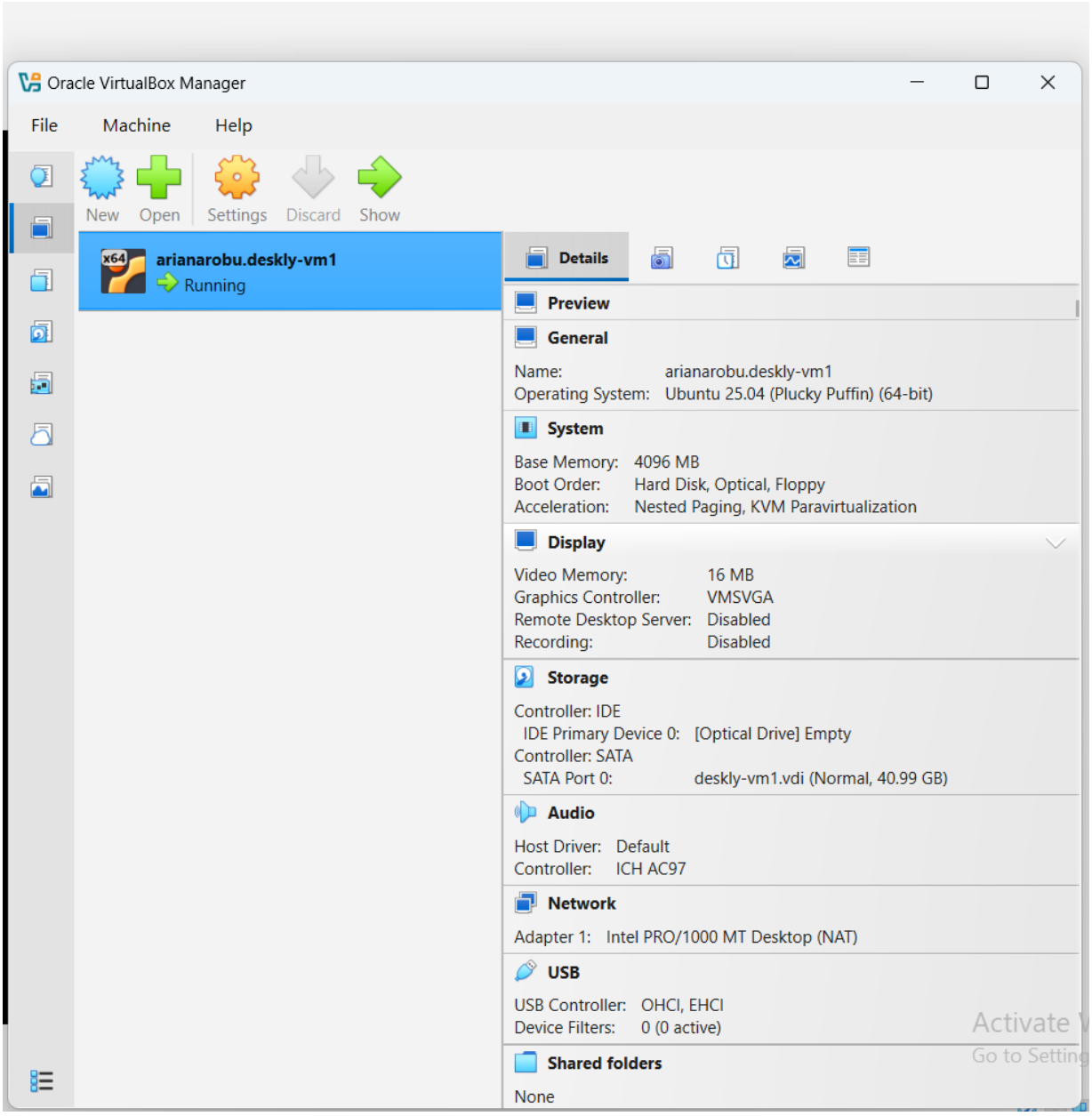
Data Layer: Baza de date SQLite cu tabelele pentru utilizatori (users), tichete (tickets), reset_tokens si log-uri (audit_logs).

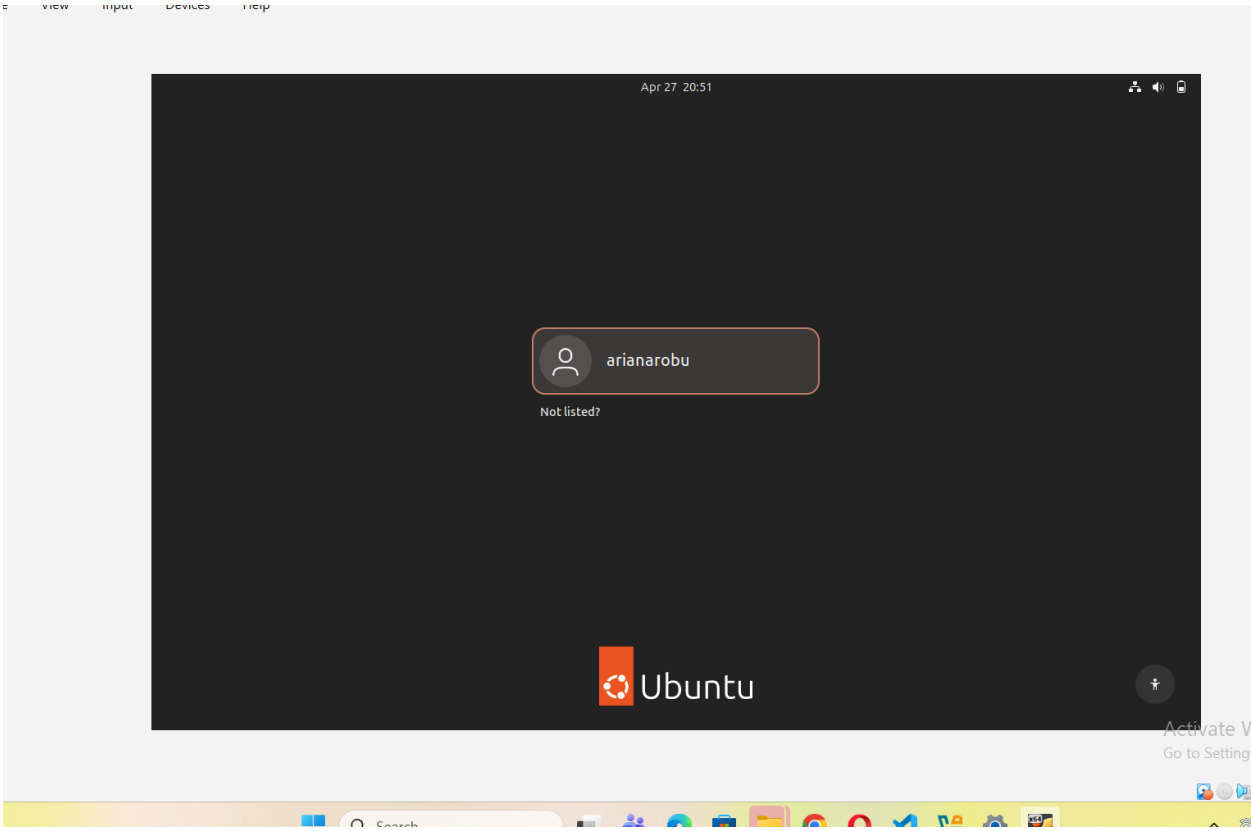


2. Setup mediu

Mediul de virtualizare

Pentru acest proiect am utilizat platforma de virtualizare Oracle VirtualBox. Am creat o masina virtuala dedicata, denumita arianarobu.deskly-vm1.


Oracle VirtualBox Manager
File
Machine
Help
New
Open
Settings
Discard
Show
arianarobu.deskly-vm1
Running
Details
Preview
General
Name: arianarobu.deskly-vm1
Operating System: Ubuntu 25.04 (Plucky Puffin) (64-bit)
System
Base Memory: 4096 MB
Boot Order: Hard Disk, Optical, Floppy
Acceleration: Nested Paging, KVM Paravirtualization
Display
Video Memory: 16 MB
Graphics Controller: VMSVGA
Remote Desktop Server: Disabled
Recording: Disabled
Storage
Controller: IDE
IDE Primary Device 0: [Optical Drive] Empty
Controller: SATA
SATA Port 0: deskly-vm1.vdi (Normal, 40.99 GB)
Audio
Host Driver: Default
Controller: ICH AC97
Network
Adapter 1: Intel PRO/1000 MT Desktop (NAT)
USB
USB Controller: OHCI, EHCI
Device Filters: 0 (0 active)
Shared folders
None



Specificatiile tehnice ale masinii virtuale sunt urmatoarele:

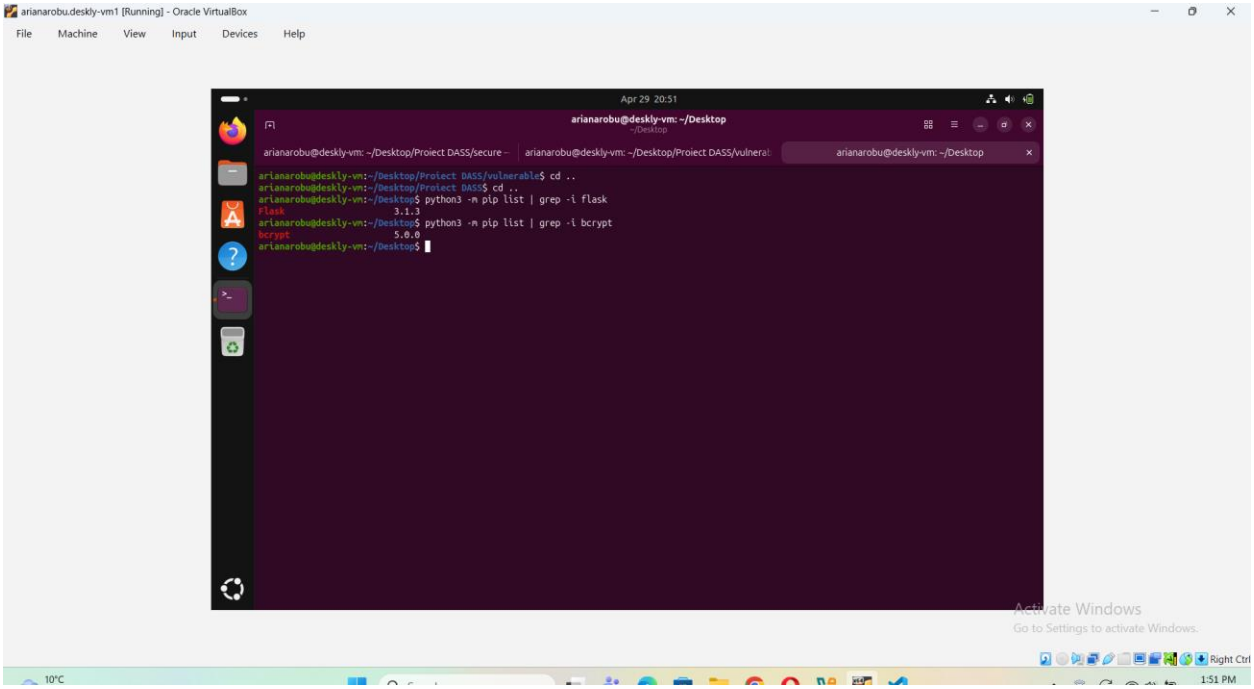
- Sistem de operare: Ubuntu 25.04 (Plucky Puffin) pe 64 de biti.
- Memorie RAM: 4096 MB.
- Stocare: Unitate virtuala VDI de aproximativ 41 GB.
- Resurse sistem: Accelerare prin KVM Paravirtualization si Nested Paging.
- Retea: Adaptor configurat in mod NAT pentru acces securizat la internet.

Componente software si biblioteci utilizate

Pentru functionarea si testarea aplicatiei, am instalat si configurat urmatoarele pachete in mediul Ubuntu:

- Python 3: Limbajul de programare in care a fost scris backend-ul aplicatiei.
- Flask: Framework-ul web utilizat pentru gestionarea serverului si a rutelor de autentificare.

- Bcrypt: Biblioteca folosita pentru hashing-ul securizat al parolelor cu salt in versiunea v2.
- SQLite3: Sistemul de gestiune a bazelor de date pentru stocarea utilizatorilor, tichetelor si log-urilor.
- Jinja2: Motorul de template-uri folosit pentru randarea paginilor HTML.
- Pip: Managerul de pachete utilizat pentru instalarea dependintelor Python.
- Curl: Unealta de linie de comanda folosita in faza de hacking pentru a trimite cereri HTTP manual catre server si pentru a analiza raspunsurile primite.



Configurarea sistemului gazda

Masina virtuala a fost instalata si configurata pe un sistem de operare gazda de tip Windows, utilizand software-ul de virtualizare Oracle VirtualBox. Aceasta structura permite rularea unui sistem de operare secundar, Ubuntu 25.04, intr-un mediu complet izolat de resursele sistemului principal.

Configurarea bazei de date

Am utilizat SQLite deoarece este o baza de date care nu necesita un server separat, fiind ideala pentru acest proiect de laborator. Am definit doua baze de date distincte:

- authx_vulnerable.db pentru versiunea v1, unde datele sunt stocate fara masuri de protectie.
- authx_secure.db pentru versiunea v2, unde am implementat tabelele users, tickets, reset_tokens si audit_logs.

Terminalul masinii virtuale a fost configurat pentru a afisa un nume de utilizator unic si hostname-ul aferent proiectului. Aceste elemente sunt vizibile in capturile de ecran din raport si permit identificarea clara a mediului de testare utilizat pe parcursul etapelor de build, hack si fix. Configuratia asigura ca toate operatiunile de sistem si executiile de cod sunt trasabile in cadrul masinii virtuale deskly-vm1.

3. Implementare MVP

Mecanismul de autentificare

Aplicatia AuthX implementeaza un flux complet de gestionare a identitatii, structurat pe urmatoarele module obligatorii conform cerintelor:

- Inregistrare utilizator: Permite crearea unui cont prin email si parola, stocand datele intr-o baza de date reala. In varianta initiala, parola este stocata nesigur, iar in varianta securizata se foloseste bcrypt.
- Autentificare: Procesul verifica credentialele in backend si creeaza o sesiune sau un token pentru a mentine utilizatorul logat.
- Logout: Asigura invalidarea sesiunii sau a token-ului si stergerea cookie-ului din browser.
- Resetare parola: Permite generarea unui token pentru setarea unei parole noi. In varianta v1 token-ul este predictibil, iar in v2 este aleatoriu si are expirare scurta.
- Gestionare sesiuni: Mentine autentificarea intre cereri si asociaza request-urile cu utilizatorul logat pentru a permite controlul accesului.

Modulul de business si tichete

Pentru a simula o aplicatie reala si pentru a testa controlul accesului, am implementat un sistem de tichete:

- Creare si editare tichete: Utilizatorii pot gestiona tichete care contin titlu, descriere, severitate si status.
- Controlul accesului: Fiecare tichet este legat de un owner_id, ceea ce permite aplicatiei sa verifice daca utilizatorul are dreptul sa acceseze resursa respectiva, prevenind atacurile de tip IDOR.

Trasabilitate si audit

Aplicatia include un modul de logare a actiunilor pentru a monitoriza activitatea utilizatorilor:

- Jurnale de audit: Se inregistreaza fiecare actiune precum login sau crearea de tichete, impreuna cu timestamp-ul si adresa IP a utilizatorului.
- Monitorizare: Aceste log-uri sunt esentiale pentru a identifica tentativele de acces neautorizat sau atacurile de tip brute force.

Structura bazei de date

Aplicatia utilizeaza SQLite pentru stocarea persistenta a datelor, avand o arhitectura formata din patru tabele corelate. Aceasta structura asigura suportul pentru autentificare, gestionarea resurselor si trasabilitatea actiunilor.

Tabelul users:

- id: Identificator unic de tip intreg, setat ca cheie primara cu autoincrement.
- email: Username-ul utilizatorului, definit ca text unic si obligatoriu.
- password_hash: Stocheaza amprenta parolei procesata prin algoritmul de hashing.
- role: Defineste nivelul de acces, avand valoarea implicita USER.
- created_at: Inregistreaza data si ora la care a fost creat contul.
- locked: Camp de tip intreg (0 sau 1) folosit pentru blocarea contului in caz de atac brute-force.

Tabelul tickets:

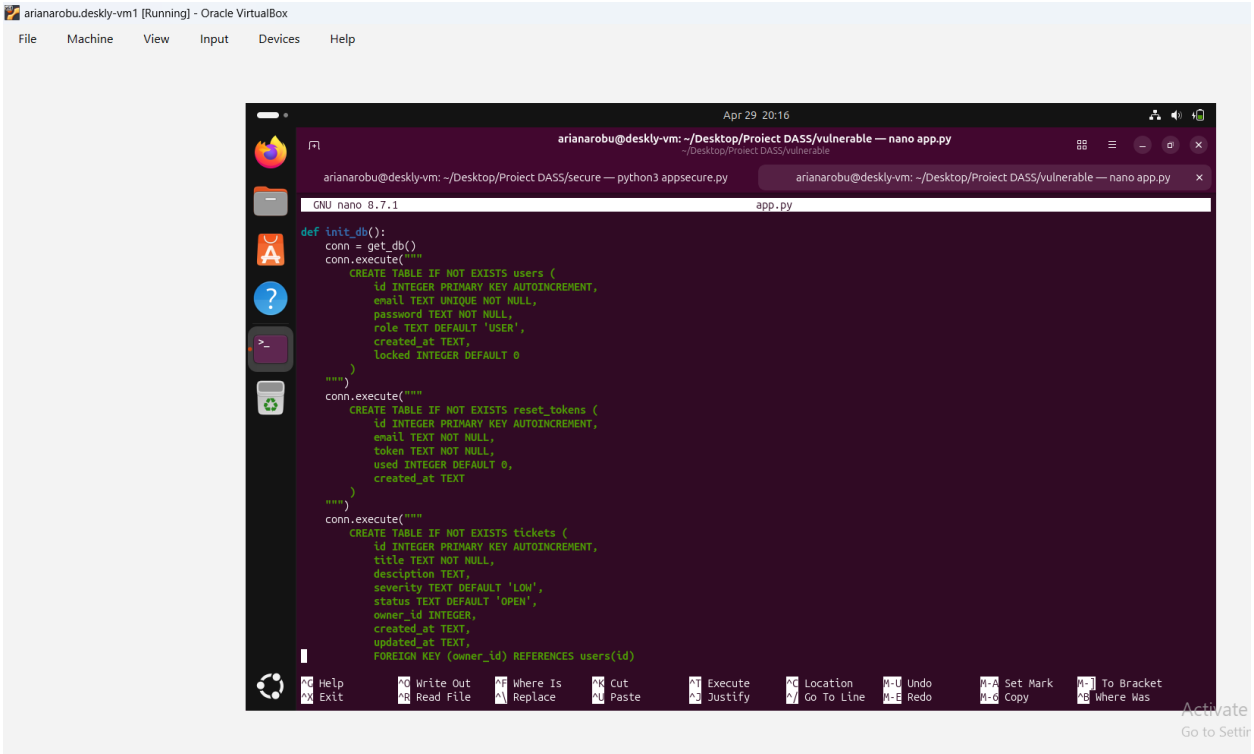
- id: Identificator unic pentru fiecare tichet creat.
- title: Titlul tichetului, marcat ca fiind obligatoriu.
- description: Detaliile tehnice ale tichetului.
- severity: Nivelul de importanta, cu valoarea implicita LOW.
- status: Starea actuala a tichetului, implicit OPEN.
- owner_id: Cheie externa catre tabelul users, folosita pentru a verifica cine detine tichetul si pentru a preveni accesul neautorizat de tip IDOR.
- created_at si updated_at: Marcaje temporale pentru crearea si modificarea resursei.

Tabelul audit_logs:

- id: Identificator pentru fiecare eveniment inregistrat.
- user_id: Cheie externa care indica utilizatorul care a efectuat actiunea.
- action: Tipul operatiunii efectuate, cum ar fi LOGIN sau LOGOUT.
- resource: Tipul de resursa accesata.
- resource_id: ID-ul specific al resursei afectate.
- timestamp: Data si ora exacta a evenimentului.
- ip_address: Adresa IP a utilizatorului, necesara pentru monitorizarea securitatii si identificarea tentativelor de atac.

Tabelul reset_tokens:

- id: Identificator unic pentru token-ul generat.
- email: Adresa asociata cererii de resetare.
- token: Sirul de caractere securizat necesar procesului de recuperare.
- used: Statusul care indica daca token-ul a fost deja utilizat (one-time use).
- created_at si expires_at: Campuri folosite pentru a valida daca token-ul se afla in fereastra de timp permisa pentru resetare.

1. 

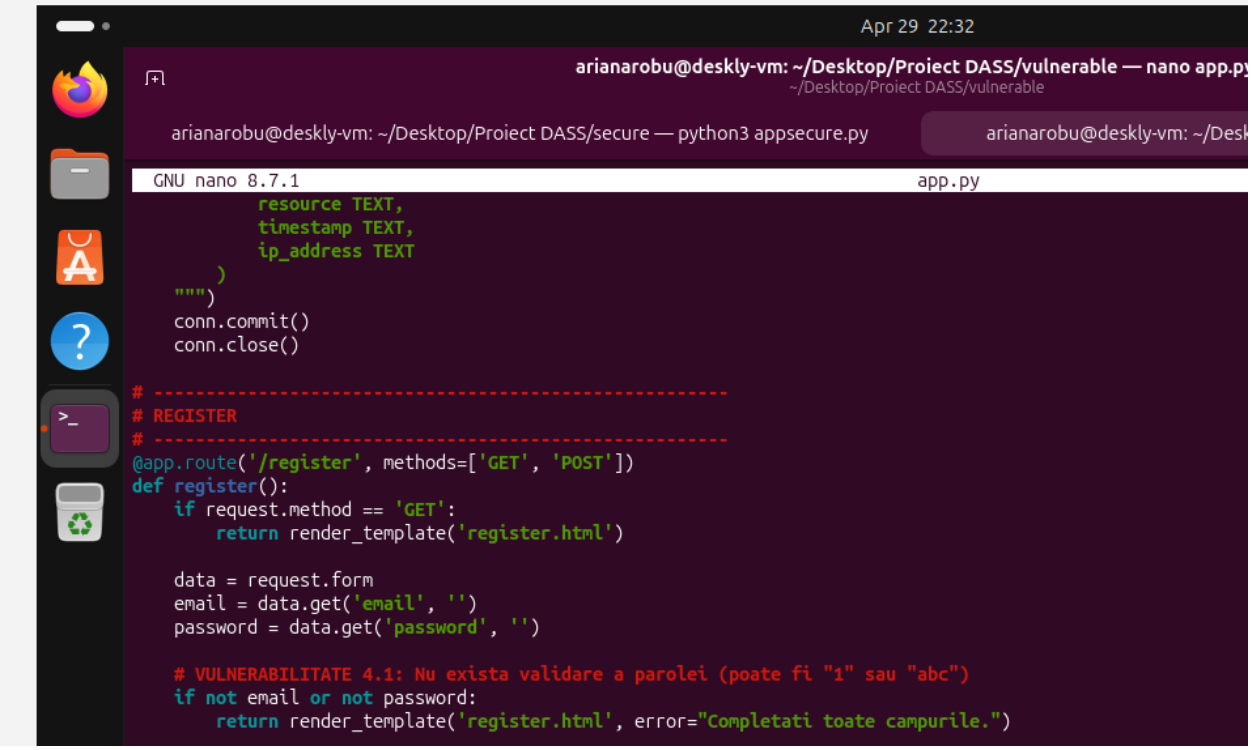
4.Prezentare Vulnerabilitati

Aceasta sectiune descrie in detaliu cele sase vulnerabilitati identificate in versiunea initiala a aplicatiei AuthX (app.py - v1 vulnerabila), cu mapare pe OWASP Top 10 si CWE.

4.1 - Politica parola slaba

In versiunea vulnerabila (app.py), nu exista nicio verificare asupra complexitatii parolei la inregistrare. Utilizatorul poate folosi parole de un singur caracter sau parole comune .

Impact: Faciliteaza atacurile de tip dictionar sau brute force.



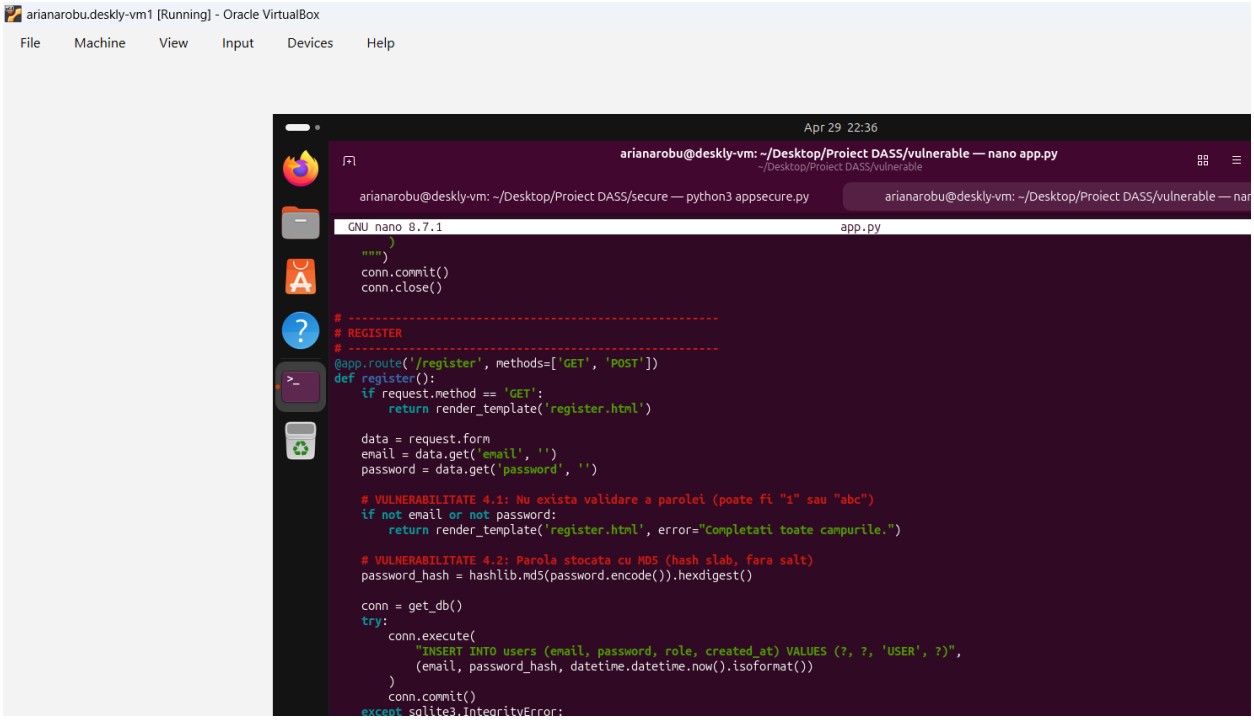
Din cod lipseste orice verificare de lungime/complexitate.

4.2 - Stocare nesigura parole

Parolele sunt stocate folosind algoritmul MD5, care este considerat nesigur si vulnerabil la atacuri de tip rainbow tables. Mai mult, nu se foloseste salt , ceea ce inseamna ca aceeasi parola va genera intotdeauna acelasi hash.

Un hash MD5 al parolei "password123" este întotdeauna același: 482c811da5d5b4bc6d497ffa98491e38. Această valoare poate fi căutată instant pe site-uri ca crackstation.net sau md5decrypt.net, fără nicio cheie secretă.

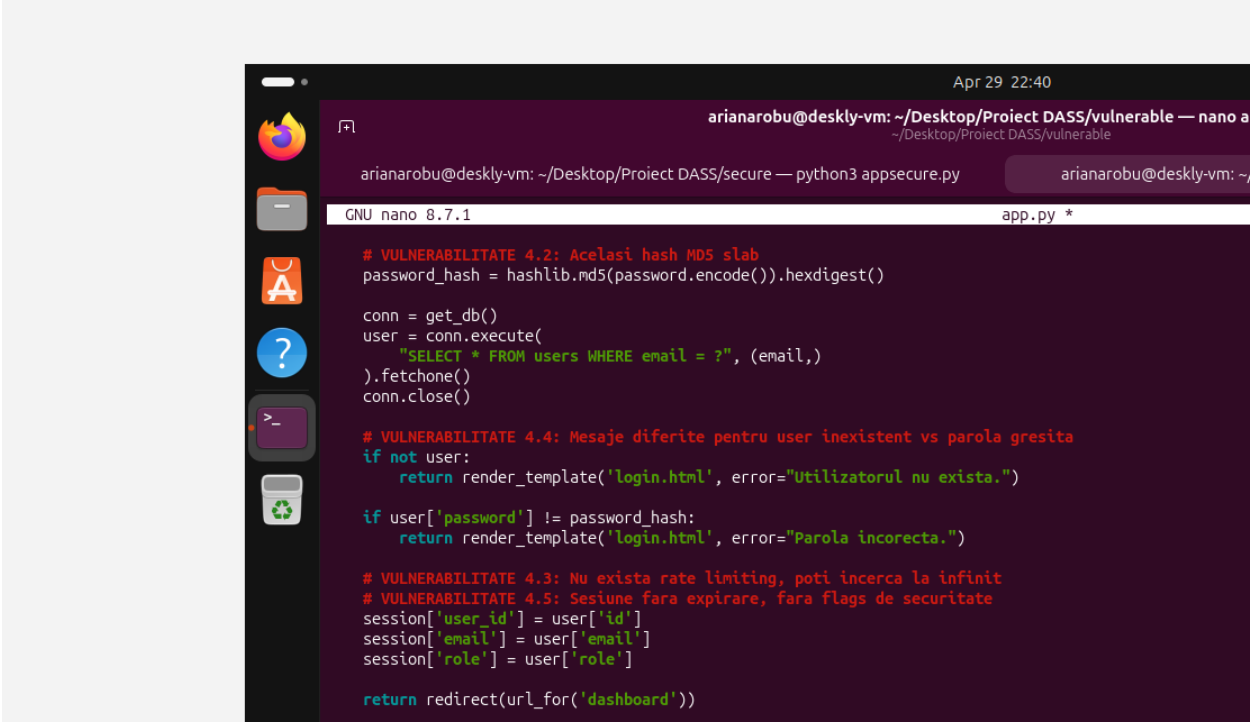
Impact: In cazul unei scurgeri de date, parolele pot fi recuperate aproape instantaneu.



4.3.Lipsa rate limiting

Endpoint-ul /login nu implementează nicio restricție privind numărul de încercări de autentificare. Un atacator poate trimite un număr nelimitat de request-uri POST cu combinații diferite de email/parolă, fără niciun fel de blocare sau întârziere.

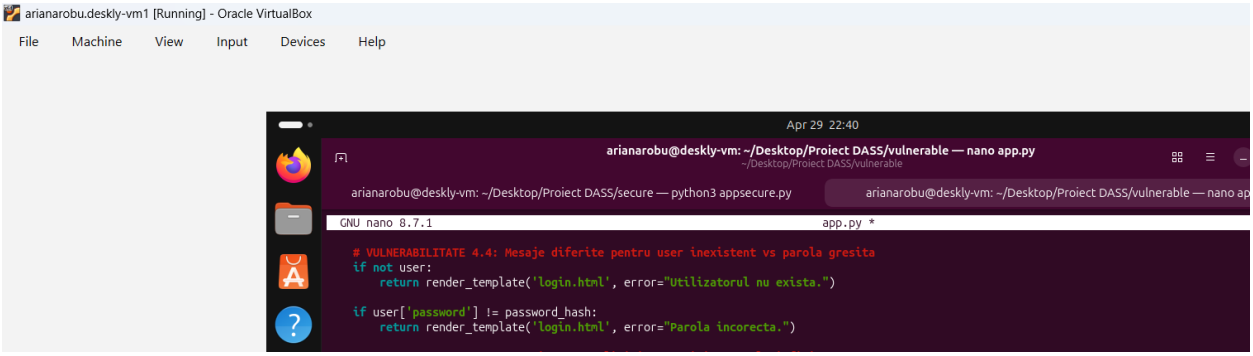
Impact: Permite spargerea conturilor prin brute force.



4.4. User enumeration

Mesajele de eroare sunt specifice: "User inexistent" vs "Parola gresita".

Impact: Un atacator poate confirma care adrese de email au cont in sistem, colectand tinte pentru phishing.



4.5. Sesiuni nesecurizate

Sesiunile create la login nu au configurate flag-urile de securitate necesare pe cookie. De asemenea, secret_key este un șir hardcodat și extrem de slab ("secret123"), ceea ce permite falsificarea sesiunilor dacă cheia este compromisă.

Vulnerabilități identificate în app.py:

secret_key = "secret123" – cheie slabă, hardcodată în sursă

Lipsă SESSION_COOKIE_HTTPONLY – permite accesul JavaScript la cookie (risc XSS)

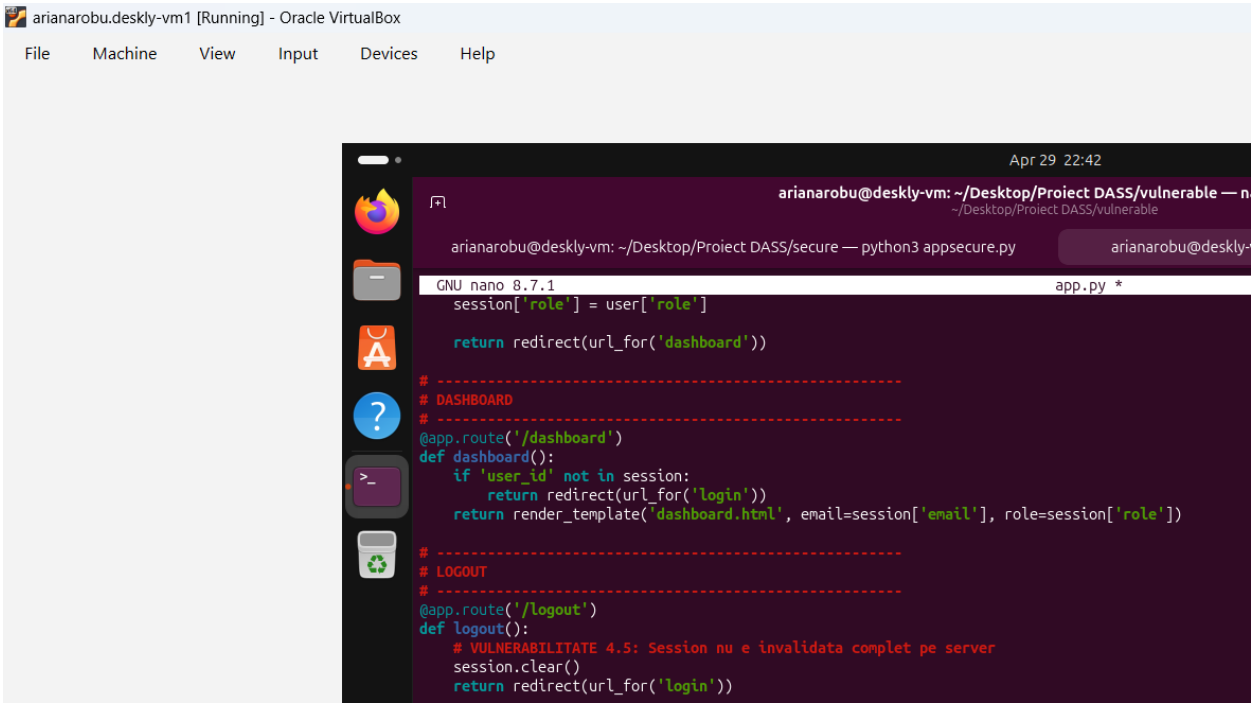
Lipsă SESSION_COOKIE_SECURE – cookie transmis și pe HTTP (neencriptat)

Lipsă SESSION_COOKIE_SAMESITE – vulnerabil la CSRF

Lipsă PERMANENT_SESSION_LIFETIME – sesiunea nu expiră niciodată

session.clear() la logout fără rotație token – posibil session fixation

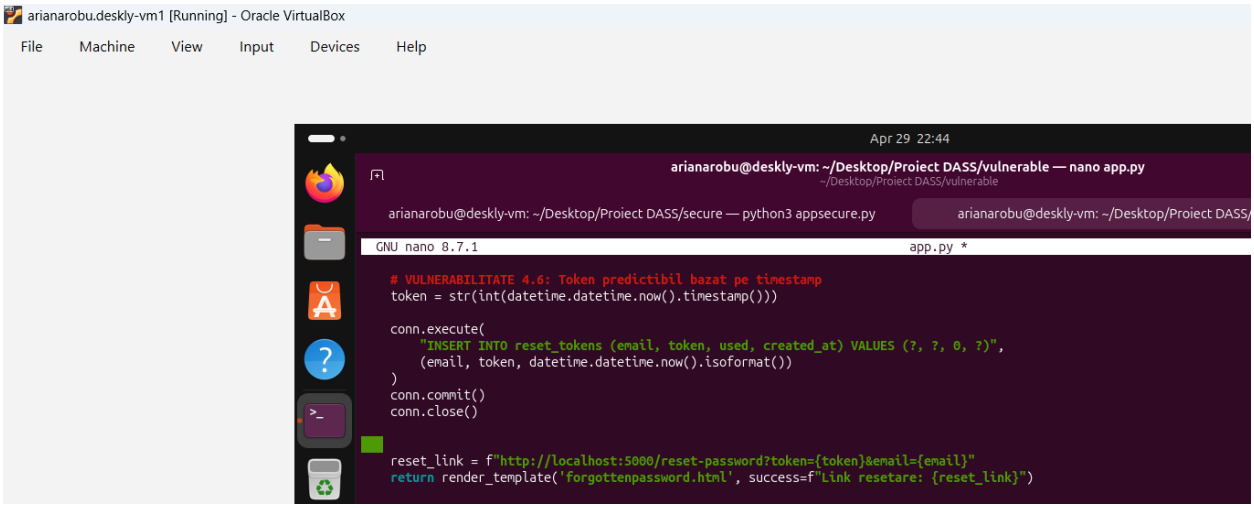
Impact: Expune sesiunea la atacuri de tip XSS (Cross-Site Scripting) si Session Hijacking.



4.6. Token resetare predictibil

Token-ul de resetare a parolei este generat pe baza unui timestamp simplu, fiind usor de ghicit de catre un script.

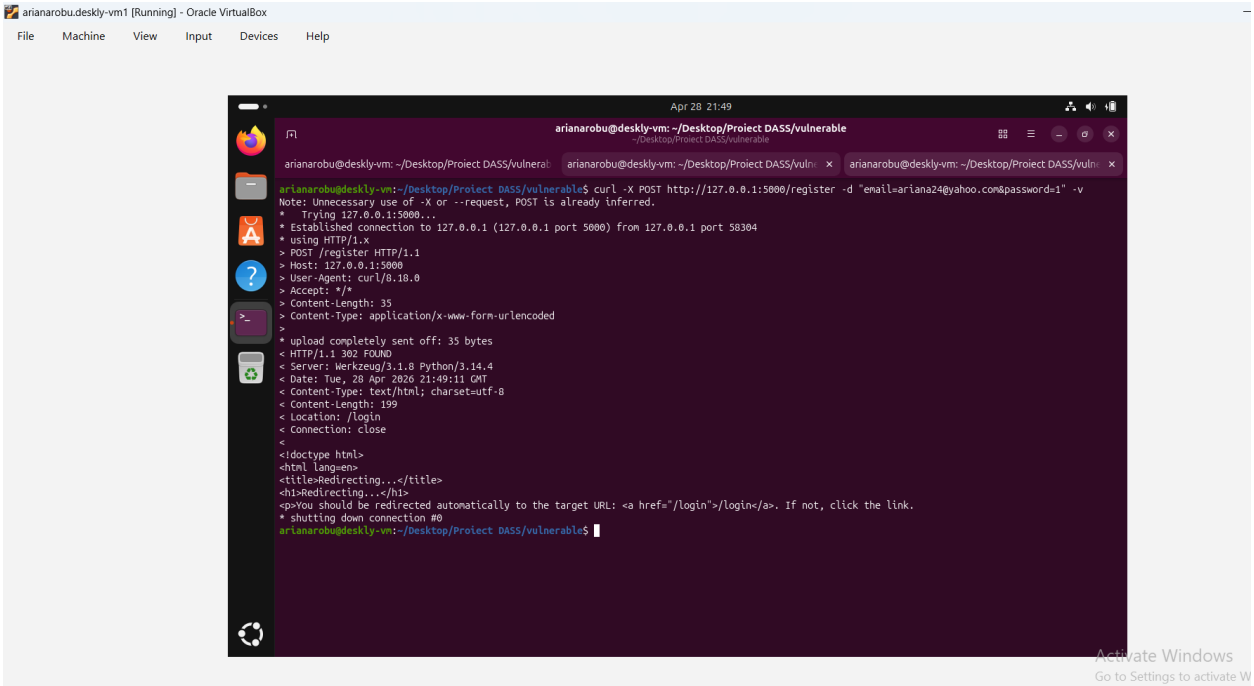
Impact: Un atacator poate genera token-uri valide pentru alti utilizatori si le poate schimba parolele fara permisiune.

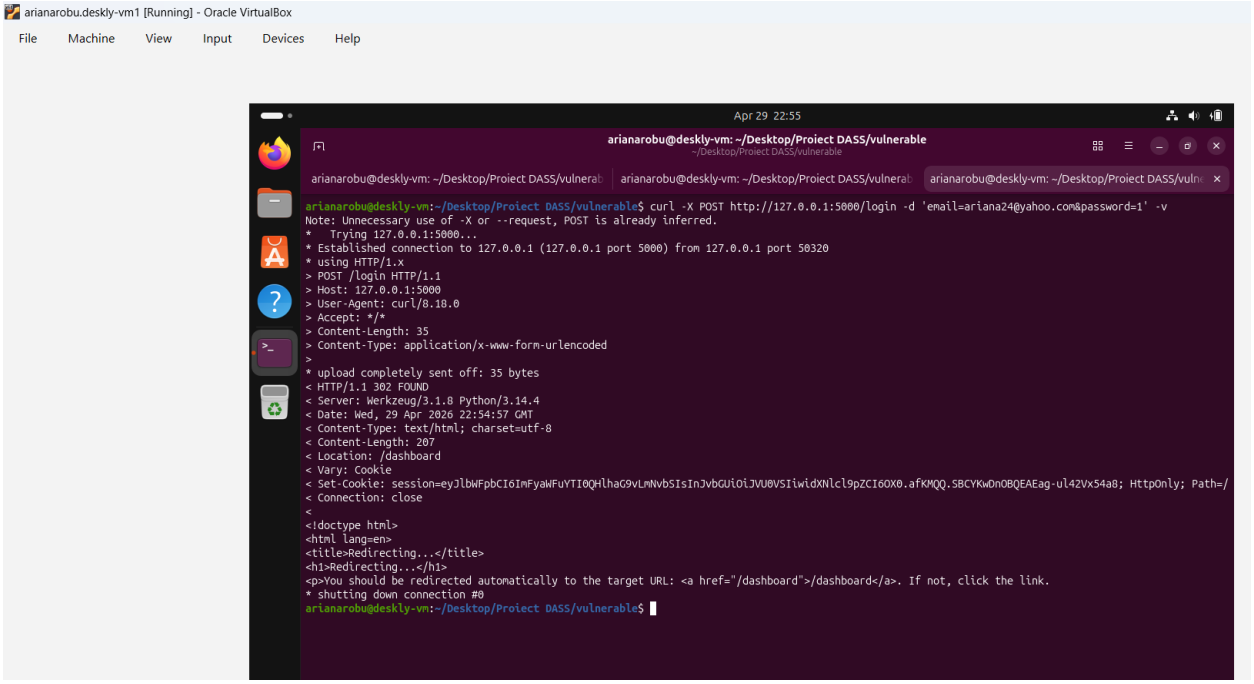


5.Demonstrare atac (PoC)

Aceasta sectiune prezinta exploatarea acestor puncte slabe folosind utilitarele de sistem.

4.1 .Creare cont cu parola slaba





Pași de exploatare

Step 1 – Trimitem o cerere de înregistrare cu parola "1"

Step 2 – Verificăm că înregistrarea a reușit

Step 3 – Ne autentificăm cu parola "1"

Rezultat observat:Aplicația acceptă parola "1" atât la înregistrare cât și la autentificare. Nu există nicio validare server-side a complexității parolei. Contul este creat și autentificarea reușește.

Impact:Utilizatorii pot crea conturi cu parole triviale, ceea ce face sistemul extrem de vulnerabil la atacuri de tip dictionary attack și credential stuffing.

4.2 . Recuperare parole din baza de date (MD5 crack)

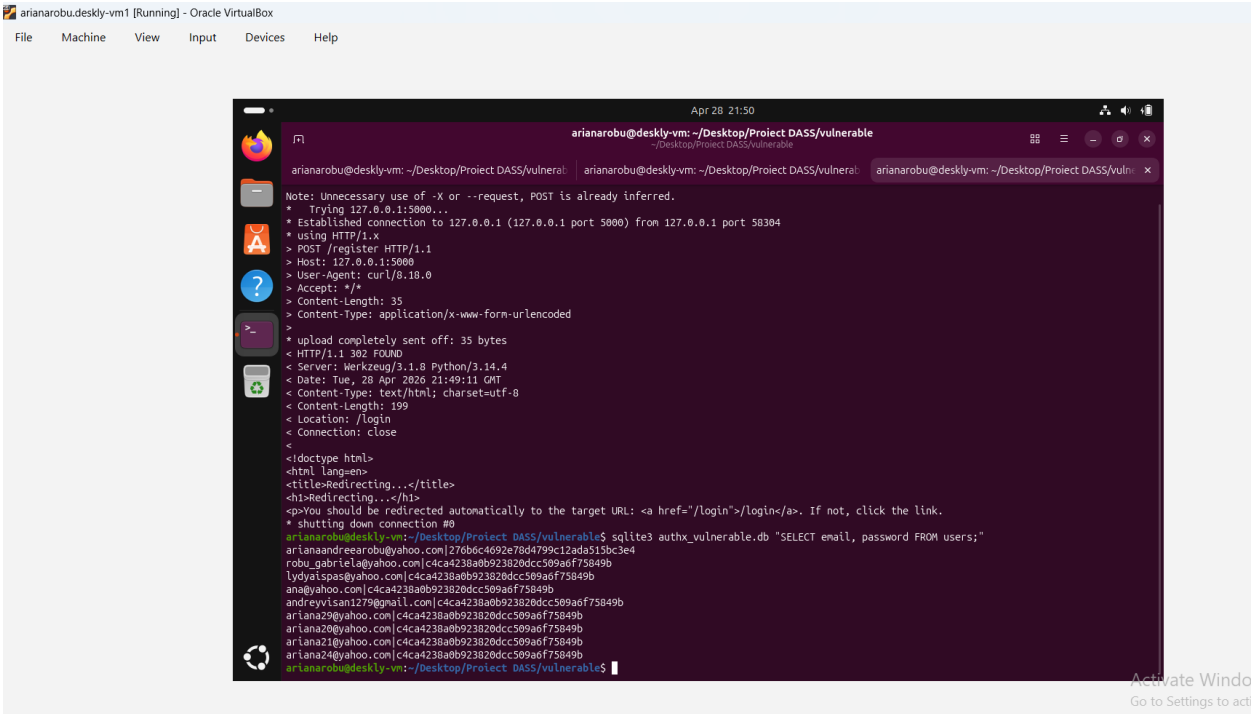
Pași de exploatare

Step 1 – Extragem hash-urile din baza de date

Step 2 – Output-ul arată hash-uri MD5 de forma

Step 3 – Căutăm hash-urile pe crackstation.net sau folosim hashcat

Step 4 – Rezultat instant pentru parole comune.



Rezultat observat:Toate parolele comune sunt recuperate în secunde fara niciun calcul semnifcativ, datorita tabelelor rainbow precomputate pentru MD5.

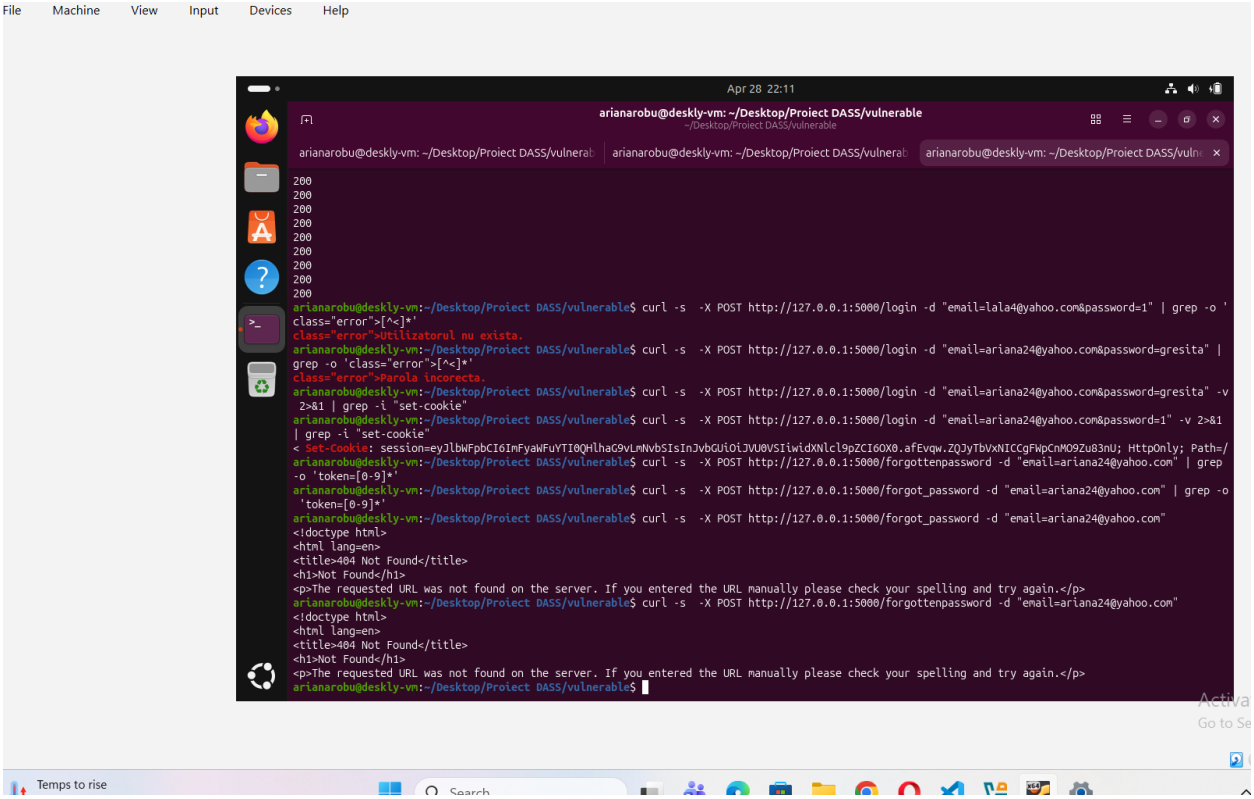
Impact:Un atacator cu acces la baza de date recuperează instant toate parolele în plain-text folosind tabele rainbow pentru MD5. Absenta salt-ului expune simultan toate conturile deoarece hash-urile identice tradează parole identice. Rezultat: compromiterea completa a tuturor utilizatorilor dintr-o singură operatie.

4.3.Atac brute force pe endpoint-ul/login

10 request-uri POST consecutive către /login returnează toate HTTP 200, fara nicio blocare sau răspuns 429 Too Many Requests. Serverul nu implementează niciun mecanism de rate limiting — un atacator poate testa parole la infinit.

Impact: Un atacator poate testa mii de parole pe secunda fara nicio restricție. Combinat cu vulnerabilitatea 4.1 (parole slabe) și 4.2 (MD5), orice cont poate fi compromis in cateva minute folosind un simplu script și o lista de parole comune.

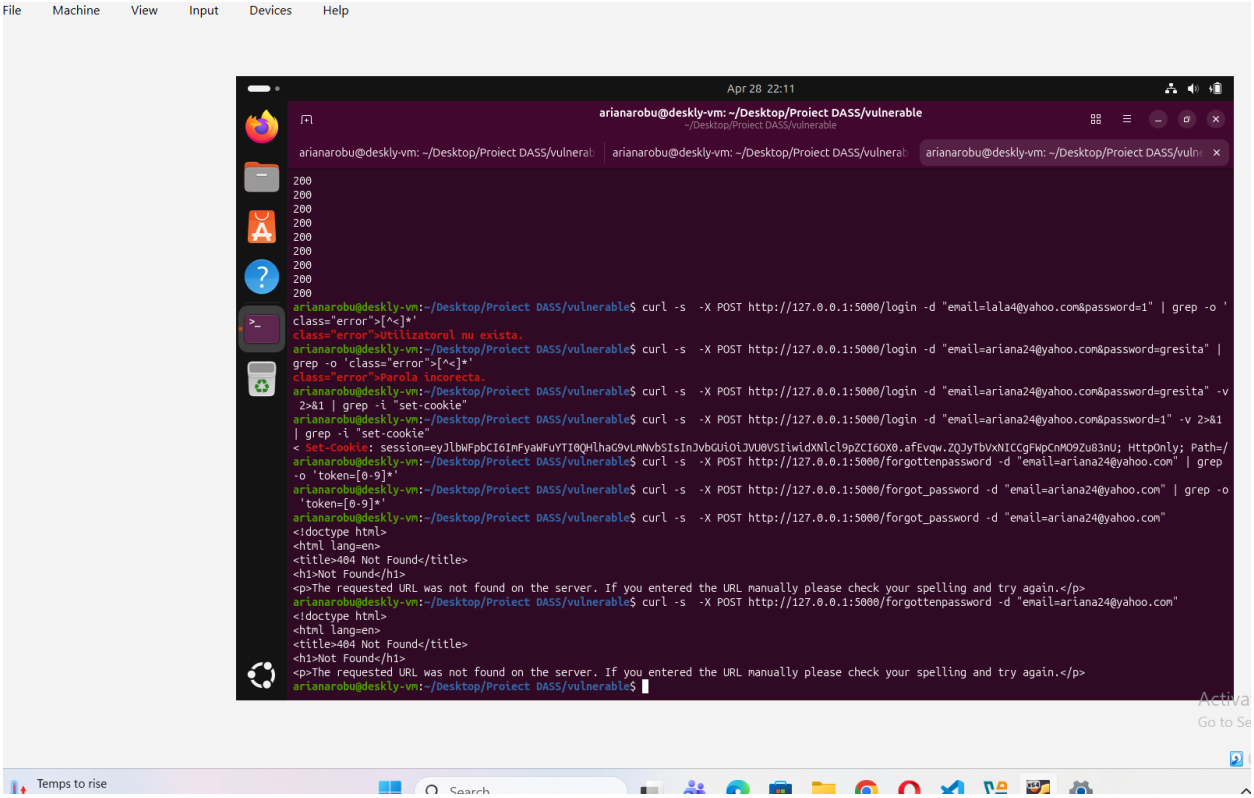
4.4.User Enumeration



Doua request-uri identice catre /login cu parole gresite returneaza mesaje diferite: email inexistent (lala4@yahoo.com) → 'Utilizatorul nu exista.', email valid (ariana24@yahoo.com) → 'Parola incorecta.'. Diferenta de mesaj permite enumerarea sistematica a utilizatorilor inregistrati in aplicatie.

Impact: Un atacator poate itera o lista de emailuri si identifica care sunt inregistrate in aplicatie doar analizand mesajul de eroare, fara a cunoaste nicio parola. Lista obtinuta poate fi folosita ulterior pentru phishing tintit sau atacuri de tip credential stuffing.

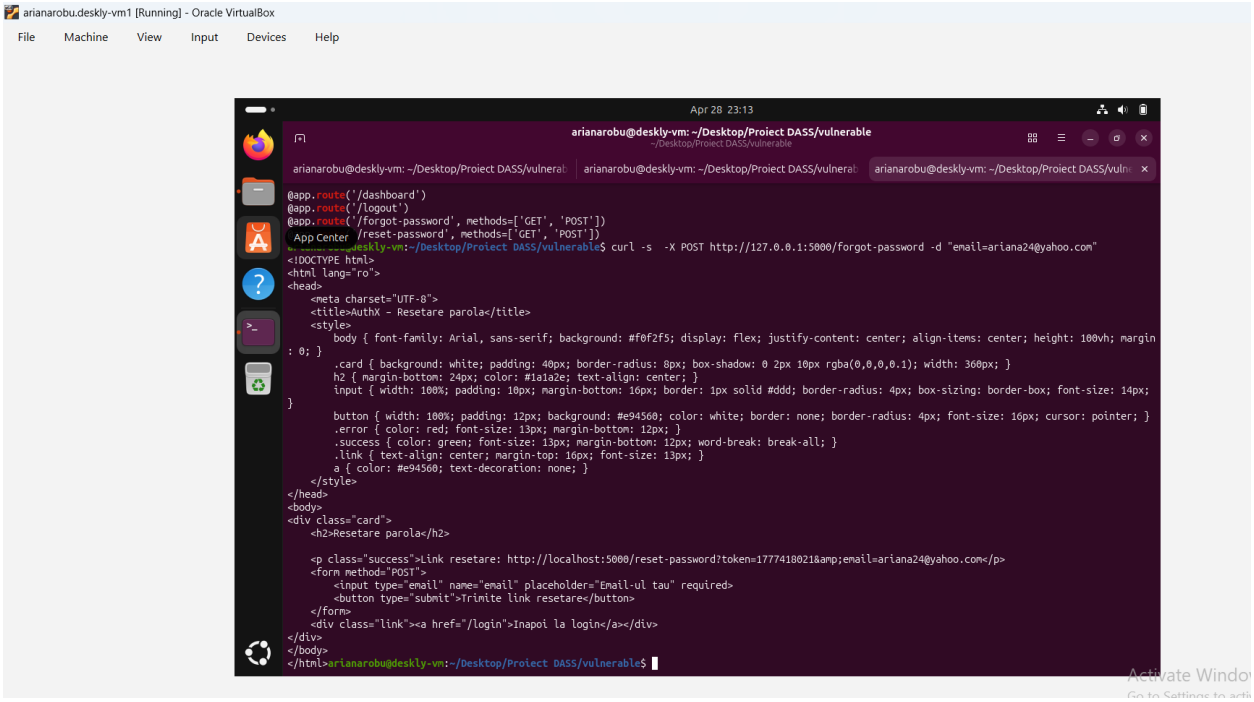
4.5.Gestionare nesigura a sesiunilor



Cookie-ul de sesiune returnat dupa autentificare contine flag-ul HttpOnly, insa lipsesc Secure si SameSite. Absenta Secure permite transmiterea cookie-ului pe conexiuni HTTP neencriptate. Absenta SameSite expune aplicatia la atacuri CSRF. Nu este definita o durata de expirare, sesiunea ramanand valida indefinit.

Impact: Un atacator care intercepteaza traficul HTTP poate fura cookie-ul de sesiune si prelua contul victimei fara a cunoaste parola (session hijacking). In prezenta unei vulnerabilitati XSS, cookie-ul poate fi exfiltrat prin JavaScript, deoarece flag-ul Secure nu impiedica accesul in afara HTTPS.

4.6.Resetare parola nesigura



Tokenul generat pentru resetarea parolei este 1777418021, reprezentand exact timestamp-ul Unix al momentului cererii . Un atacator care cunoaste aproximativ momentul cererii poate genera toate valorile posibile intr-o fereastra de +-60 secunde (121 tokene) si identifica tokenul valid prin testare automata.

Impact: Un atacator poate prelua orice cont fara a cunoaste parola, doar prin ghicirea tokenului de resetare. Deoarece tokenul nu este marcat ca folosit dupa utilizare, acesta poate fi reutilizat de nenumarate ori. Combinat cu user enumeration (4.4), atacatorul stie exact ce emailuri sunt valide si poate tinta conturi specifice.

Analiza Impact

6.Analiza Impact

Combinarea vulnerabilitatilor descrise creeaza un lant de atac complet asupra sistemului AuthX. Mai jos este prezentat ce poate obtine un atacator pentru fiecare vulnerabilitate identificata:

Parola slaba (4.1): Un atacator poate crea conturi cu parole triviale, facilitand brute force cu liste standard de parole comune.

MD5 fara salt (4.2): Recuperare parole plain-text din baza de date prin tabele rainbow. Un dump al DB expune instant toate parolele utilizatorilor.

Fara rate limit (4.3): Testare mii de parole pe secunda fara nicio restrictie. Un script simplu sparge orice cont in cateva minute.

User enumeration (4.4): Construirea unei liste complete de utilizatori valizi, folosita ulterior pentru phishing tintit si credential stuffing.

Sesiuni nesecurizate (4.5): Furt sesiune prin interceptare HTTP sau XSS, permitand session hijacking fara a cunoaste parola.

Token resetare predictibil (4.6): Preluarea oricarui cont prin ghicirea tokenului de resetare, fara interactiunea victimei.

Scenariul cel mai sever (attack chain): Un atacator foloseste user enumeration (4.4) pentru a identifica emailuri valide, le crackuieste parolele MD5 (4.2) dupa un eventual dump de DB, sau foloseste brute force fara rate limiting (4.3). Alternativ, foloseste tokenul de resetare predictibil (4.6) pentru a prelua orice cont fara a cunoaste parola.

7.Implementare Fix (v2 securizat)

Versiunea securizata este implementata in appsecure.py.Fiecare fix corespunde direct vulnerabilitatii identificate si elimina vectorul de atac demonstrat.

Fix 4.1.Validare complexitate parola

Am implementat o funcție de validare validate_password() care este apelata atat la înregistrare cat și la resetarea parolei:

arianarobu.deskly-vm1 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Apr 29 23:35

arianarobu@deskly-vm: ~/Desktop/Proiect DASS/secure — nano appsecure.py

~/Desktop/Proiect DASS/secure

arianarobu@deskly-vm: ~/Desktop/Proiect DASS/vulnerable

arianarobu@deskly-vm: ~/Desktop/Proiect DASS

GNU nano 8.7.1 appsecure.py

```
# ----------------------------------------------------
# FIX 4.1: Validare parola
# ----------------------------------------------------
def validate_password(password):
    """
    Returneaza (True, None) daca parola e valida,
    sau (False, "mesaj eroare") daca nu.
    """
    if len(password) < 8:
        return False, "Parola trebuie sa aiba minim 8 caractere."
    if not any(c.isupper() for c in password):
        return False, "Parola trebuie sa contina cel putin o litera mare."
    if not any(c.isdigit() for c in password):
        return False, "Parola trebuie sa contina cel putin o cifra."
    if not any(c in "!@#$%^&*()_+-=[]{}|;:,.<>?" for c in password):
        return False, "Parola trebuie sa contina cel putin un caracter special."
    return True, None
```

arianarobu.deskly-vm1 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Apr 29 23:44

arianarobu@deskly-vm: ~/Desktop/Proiect DASS/secure — nano appsecure.py

~/Desktop/Proiect DASS/secure

arianarobu@deskly-vm: ~/Desktop/Proiect DASS/vulnerable

arianarobu@deskly-vm: ~/Desktop/Proiect D

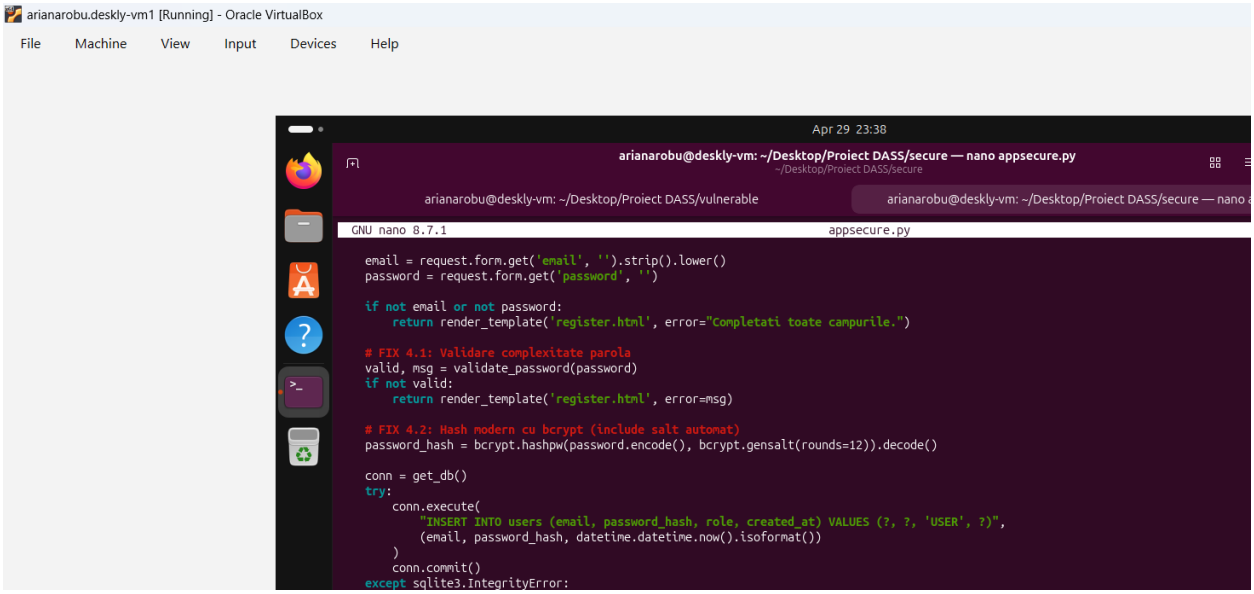
GNU nano 8.7.1 appsecure.py *

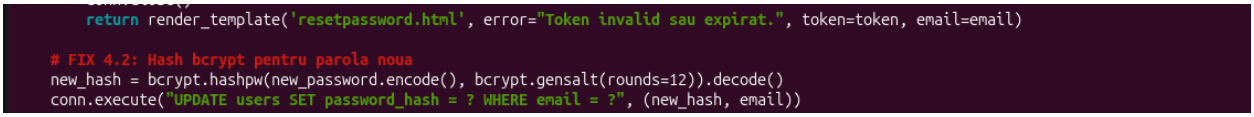
```
        )
        conn.commit()

        # In productie: trimite pe email. In lab: afisam link-ul.
        reset_link = f"http://localhost:5000/reset-password?token={token}&email={email}"
        success_msg = f"Link resetare (doar in lab): {reset_link}"

    conn.close()
    return render_template('forgottenpassword.html', success=success_msg)

# ----------------------------------------------------
# RESET PASSWORD
# ----------------------------------------------------
@app.route('/reset-password', methods=['GET', 'POST'])
def reset_password():
    token = request.args.get('token', '')
    email = request.args.get('email', '').strip().lower()

    if request.method == 'GET':
        return render_template('resetpassword.html', token=token, email=email)

    new_password = request.form.get('password', '')
    token = request.form.get('token', '')
    email = request.form.get('email', '').strip().lower()

    # FIX 4.1: Validam si parola noua
    valid, msg = validate_password(new_password)
    if not valid:
        return render_template('resetpassword.html', error=msg, token=token, email=email)

    conn = get_db()
    now = datetime.datetime.now().isoformat()
```

Reguli implementate: minim 8 caractere, cel puțin o literă mare, cel puțin o cifră, cel puțin un caracter special. Mesajele de eroare sunt specifice pentru a ghida utilizatorul, dar nu ajută un atacator.

Fix 4.2.Hash cu bcrypt

Am înlocuit hashlib.md5() cu bcrypt, care include salt automat și un factor de cost configurabil (rounds=12):





bcrypt cu rounds=12 generează un hash diferit pentru aceeași parolă la fiecare apel (salt random inclus). Nu există tabele rainbow și calculul unui hash durează ~250ms deliberat, făcând brute force extrem de costisitor.

Fix 4.3.Rate limiting si blocare cont

Am implementat un mecanism in-memory de rate limiting bazat pe IP-ul clientului. Maximum 5 încercări în 5 minute per IP:

arianarobu.deskly-vm1 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Apr 29 23:42

arianarobu@deskly-vm: ~/Desktop/Proiect DASS/secure — nano appsecure.py

~/Desktop/Proiect DASS/secure

arianarobu@deskly-vm: ~/Desktop/Proiect DASS/vulnerable | arianarobu@deskly-vm: ~/Desktop/Proiect DASS/secure —

GNU nano 8.7.1 appsecure.py

```
    """
    if len(password) < 8:
        return False, "Parola trebuie sa aiba minim 8 caractere."
    if not any(c.isupper() for c in password):
        return False, "Parola trebuie sa contina cel putin o litera mare."
    if not any(c.isdigit() for c in password):
        return False, "Parola trebuie sa contina cel putin o cifra."
    if not any(c in "!@#$%^&*()_+-=[]{}|;:,.<>?" for c in password):
        return False, "Parola trebuie sa contina cel putin un caracter special."
    return True, None

# ----------------------------------------------------
# FIX 4.3: Rate limiting
# ----------------------------------------------------
def is_rate_limited(ip):
    now = time.time()
    attempts = login_attempts.get(ip, [])
    # Pastreaza doar incercarile din ultimele LOCKOUT_SECONDS secunde
    attempts = [t for t in attempts if now - t < LOCKOUT_SECONDS]
    login_attempts[ip] = attempts
    return len(attempts) >= MAX_ATTEMPTS

def record_attempt(ip):
    now = time.time()
    attempts = login_attempts.get(ip, [])
    attempts.append(now)
    login_attempts[ip] = attempts
```

arianarobu.deskly-vm1 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Apr 29 23:42

Firefox

arianarobu@deskly-vm: ~/Desktop/Proiect DASS/secure — nano appsecure.py

~/Desktop/Proiect DASS/secure

arianarobu@deskly-vm: ~/Desktop/Proiect DASS/vulnerable | arianarobu@deskly-vm: ~/Desktop/Proiect DASS/secure —

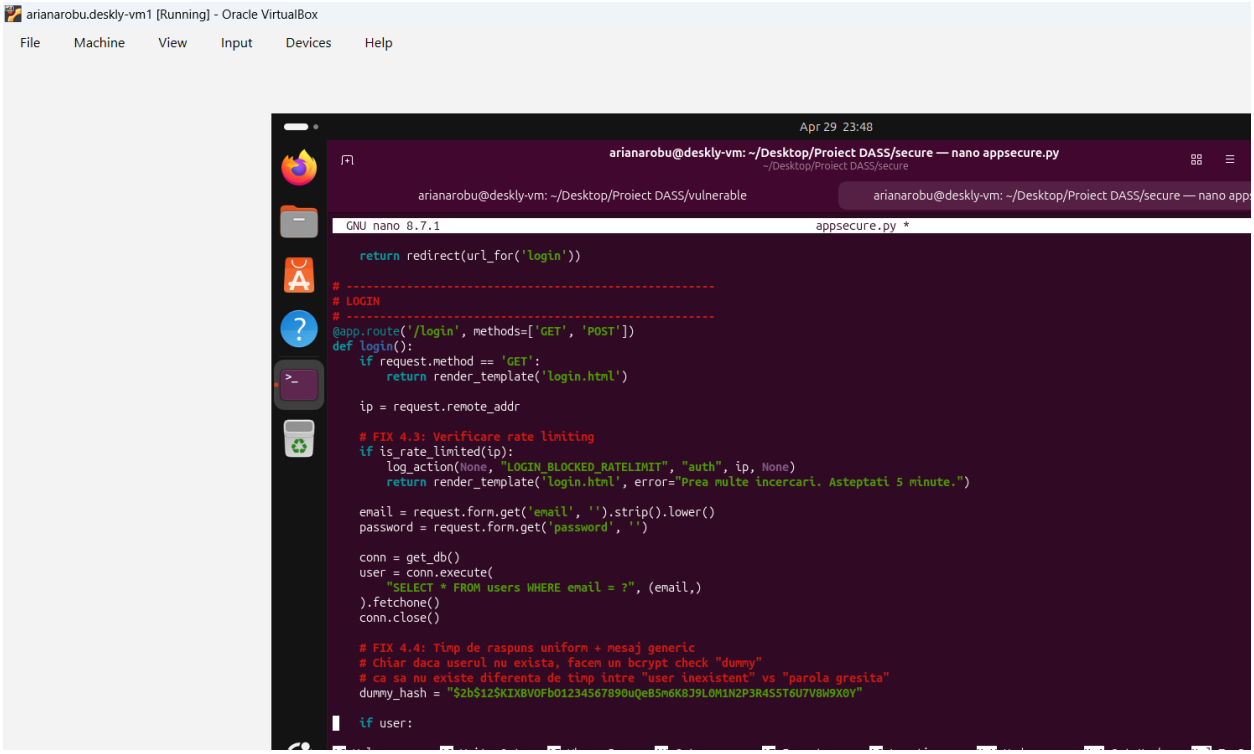
GNU nano 8.7.1 appsecure.py *

```
# ----------------------------------------------------
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'GET':
        return render_template('login.html')

    ip = request.remote_addr

    # FIX 4.3: Verificare rate limiting
    if is_rate_limited(ip):
        log_action(None, "LOGIN_BLOCKED_RATELIMIT", "auth", ip, None)
        return render_template('login.html', error="Prea multe incercari. Asteptati 5 minute.")

    email = request.form.get('email', '').strip().lower()
    password = request.form.get('password', '')

    conn = get_db()
    user = conn.execute(
        "SELECT * FROM users WHERE email = ?", (email,)
    ).fetchone()
    conn.close()
```

Fiecare tentativă eșuată este înregistrată. La depășirea limitei, IP-ul este blocat 5 minute și tentativa este logată în audit_logs cu acțiune LOGIN_BLOCKED_RATELIMIT.

Fix 4.4.Eliminare user enumeration

Am unificat toate mesajele de eroare la un mesaj generic și am adăugat un bcrypt check "dummy" pentru a uniformiza și timpul de raspuns:



Același mesaj "Credentiale invalide." este returnat indiferent dacă emailul nu exista sau parola e gresita. Timing-ul de raspuns este uniform în ambele cazuri.

Fix 4.5.Securizare sesiuni

Am configurat Flask cu toți parametrii de securitate necesari pentru cookie-ul de sesiune:

arianarobu.deskly-vm1 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Apr 29 23:50

arianarobu@deskly-vm: ~/Desktop/Proiect DASS/secure — nano appsecure.py

~/Desktop/Proiect DASS/secure

arianarobu@deskly-vm: ~/Desktop/Proiect DASS/vulnerable | arianarobu@deskly-vm: ~/Desktop/Proiect DASS/secure —

GNU nano 8.7.1 appsecure.py *

```
from flask import Flask, request, jsonify, session, render_template, redirect, url_for
import sqlite3
import bcrypt
import os
import secrets
import datetime
import time
from functools import wraps

app = Flask(__name__)

# FIX 4.5: Secret key puternica, citita din variabila de mediu (nu hardcodata)
app.secret_key = os.environ.get("SECRET_KEY", secrets.token_hex(32))

# FIX 4.5: Configurare sesiune securizata
app.config.update(
    SESSION_COOKIE_HTTPONLY=True,       # Previne accesul JavaScript la cookie
    SESSION_COOKIE_SECURE=False,        # Pune True in productie (HTTPS)
    SESSION_COOKIE_SAMESITE='Lax',      # Protectie CSRF de baza
    PERMANENT_SESSION_LIFETIME=datetime.timedelta(minutes=30)  # Expirare sesiune
)

DATABASE = "authx_secure.db"
```

arianarobu.deskly-vm1 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Apr 29 23:51

arianarobu@deskly-vm: ~/Desktop/Proiect DASS/secure — nano appsecure.py

~/Desktop/Proiect DASS/secure

arianarobu@deskly-vm: ~/Desktop/Proiect DASS/vulnerable | arianarobu@deskly-vm: ~/Desktop/Proiect DASS/secure — nan

GNU nano 8.7.1 appsecure.py *

```
    # FIX 4.5: Sesiune securizata cu expirare si rotatie
    session.clear()  # Rotatie token - previne session fixation
    session.permanent = True
    session['user_id'] = user['id']
    session['email'] = user['email']
    session['role'] = user['role']
    session['created_at'] = datetime.datetime.now().isoformat()

    log_action(user['id'], "LOGIN_SUCCESS", "auth", ip, None)
    return redirect(url_for('dashboard'))
```

arianarobu.deskly-vm1 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Apr 29 23:51

arianarobu@deskly-vm: ~/Desktop/Proiect DASS/secure — nano appse

~/Desktop/Proiect DASS/secure

arianarobu@deskly-vm: ~/Desktop/Proiect DASS/vulnerable | arianarobu@deskly-vm: ~/

GNU nano 8.7.1 appsecure.py *

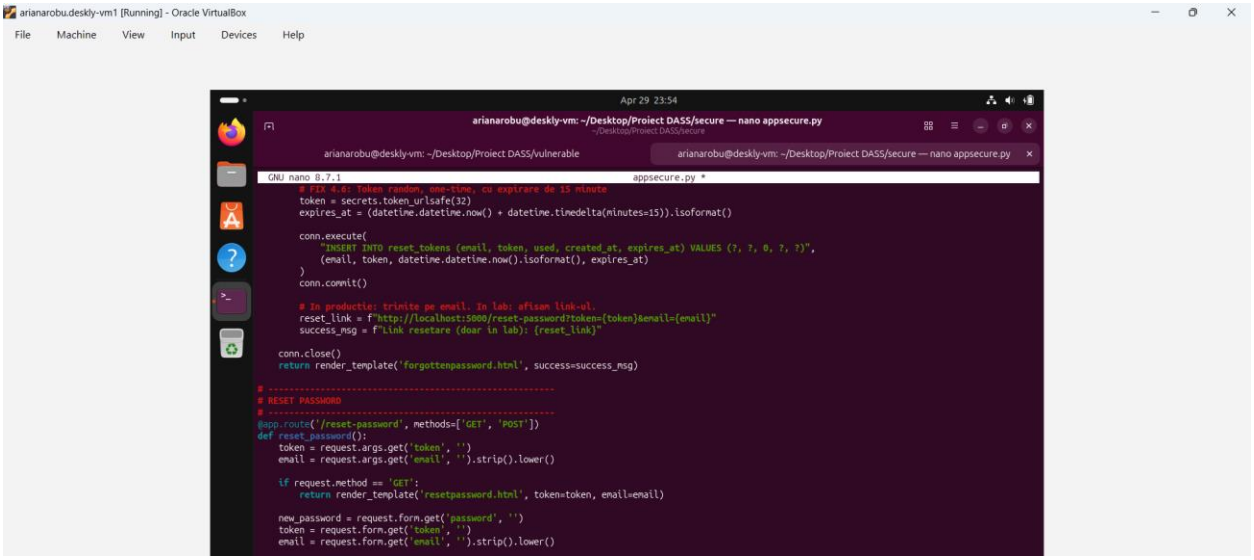
App Center

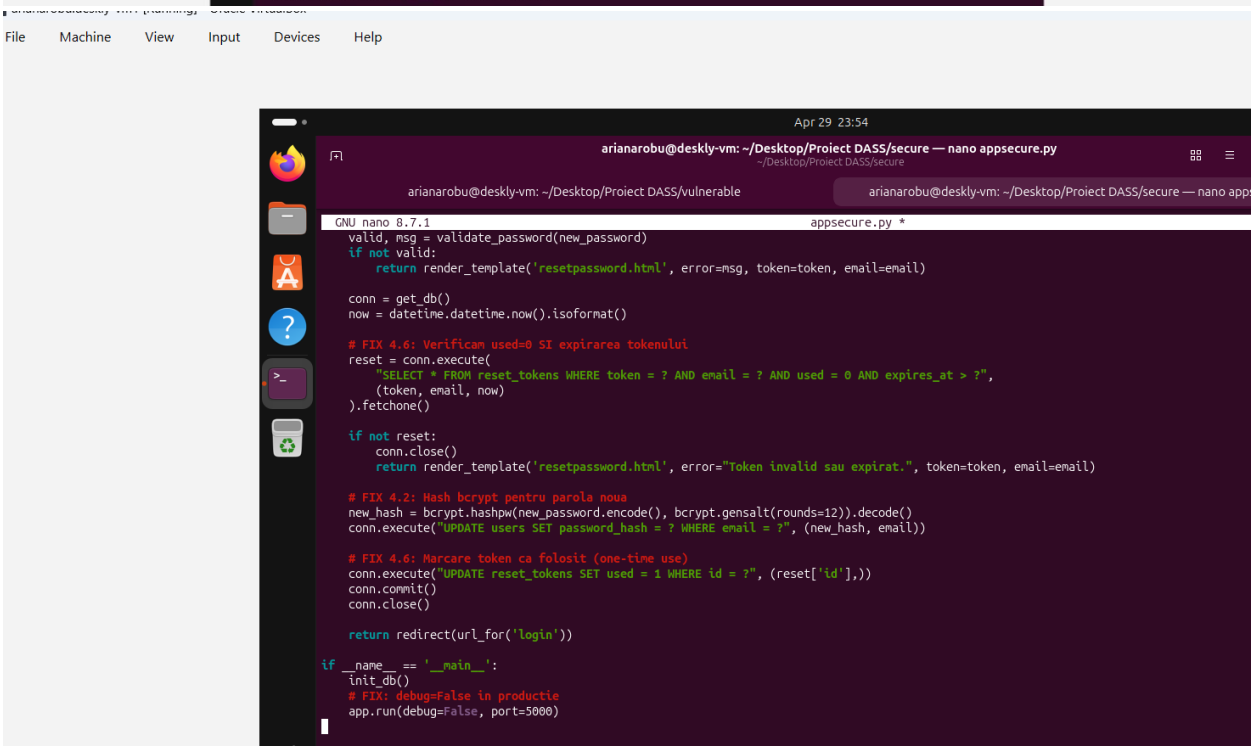
```
# ----------------------------------------------------------
# LOGOUT
# ----------------------------------------------------------
@app.route('/logout')
def logout():
    user_id = session.get('user_id')
    ip = request.remote_addr
    # FIX 4.5: Sesiunea e curatata complet
    session.clear()
    log_action(user_id, "LOGOUT", "auth", ip, None)
    return redirect(url_for('login'))
```

Secret key este citita din variabila de mediu SECRET_KEY. Dacă nu e setata, se genereaza random la fiecare pornire (32 bytes hex = 256 biți entropie).

Fix 4.6.Token resetare parola sigur

Am inlocuit tokenul bazat pe timestamp cu un token criptografic random, cu expirare de 15 minute și invalidare dupa utilizare:



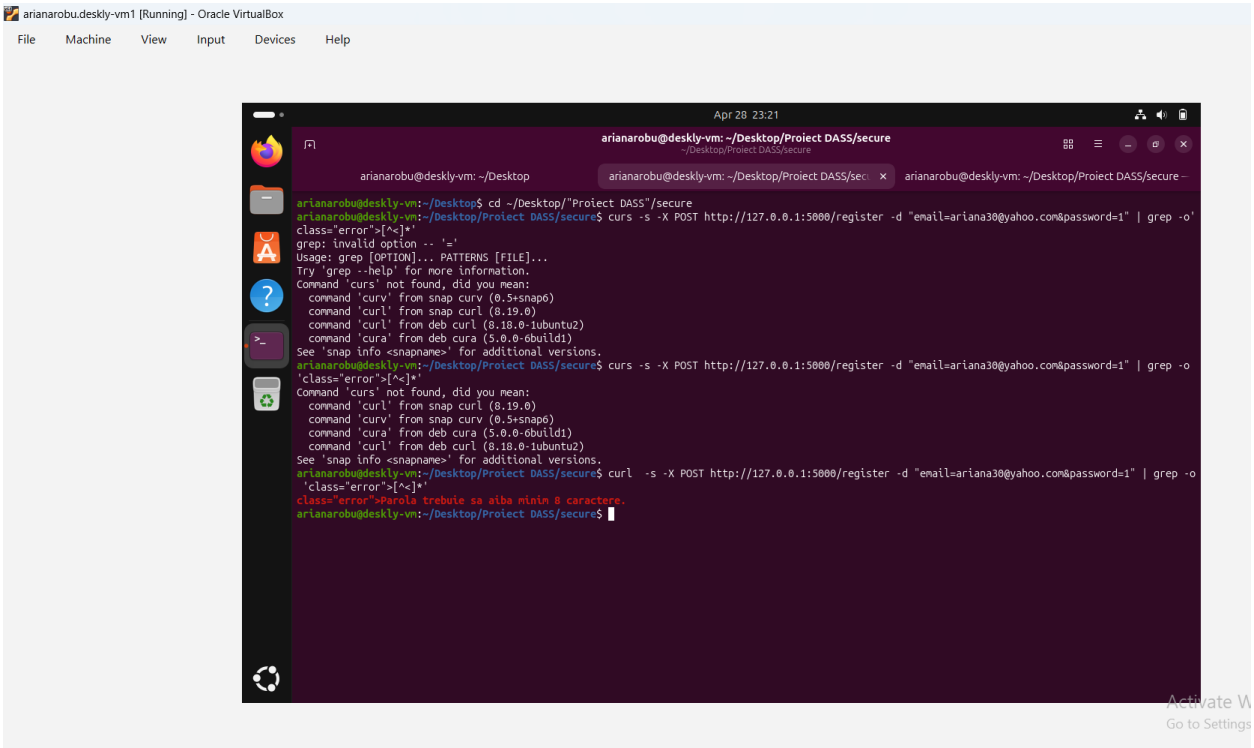


Token-ul are 32 bytes de entropie criptografica (secrets.token_urlsafe), expira după 15 minute și este invalidat imediat dupa prima utilizare reusita.

8.Re-test-Dovada ca atacurile nu mai functioneaza

Toate testele de mai jos au fost executate împotriva versiunii securizate (appsecure.py) ruland pe http://localhost:5000.

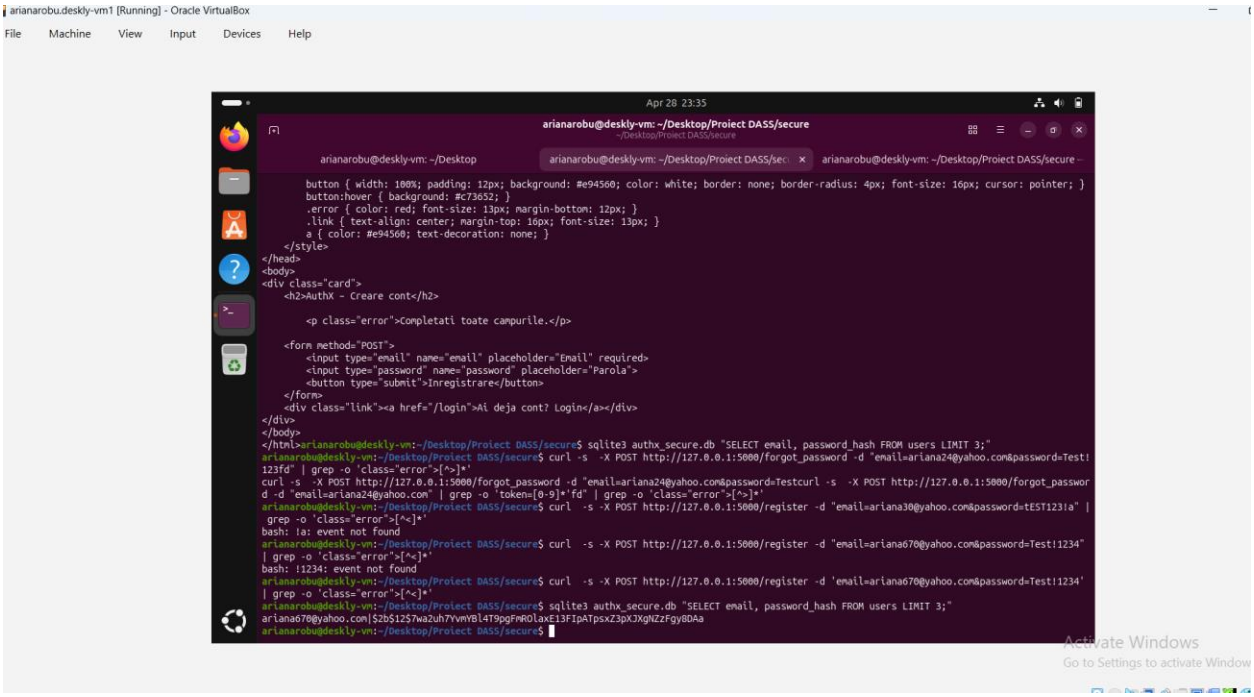
Re-test 4.1.Parola slaba



Rezultat asteptat:

Serverul returnează eroarea: "Parola trebuie sa aiba minim 8 caractere." Înregistrarea este refuzata.

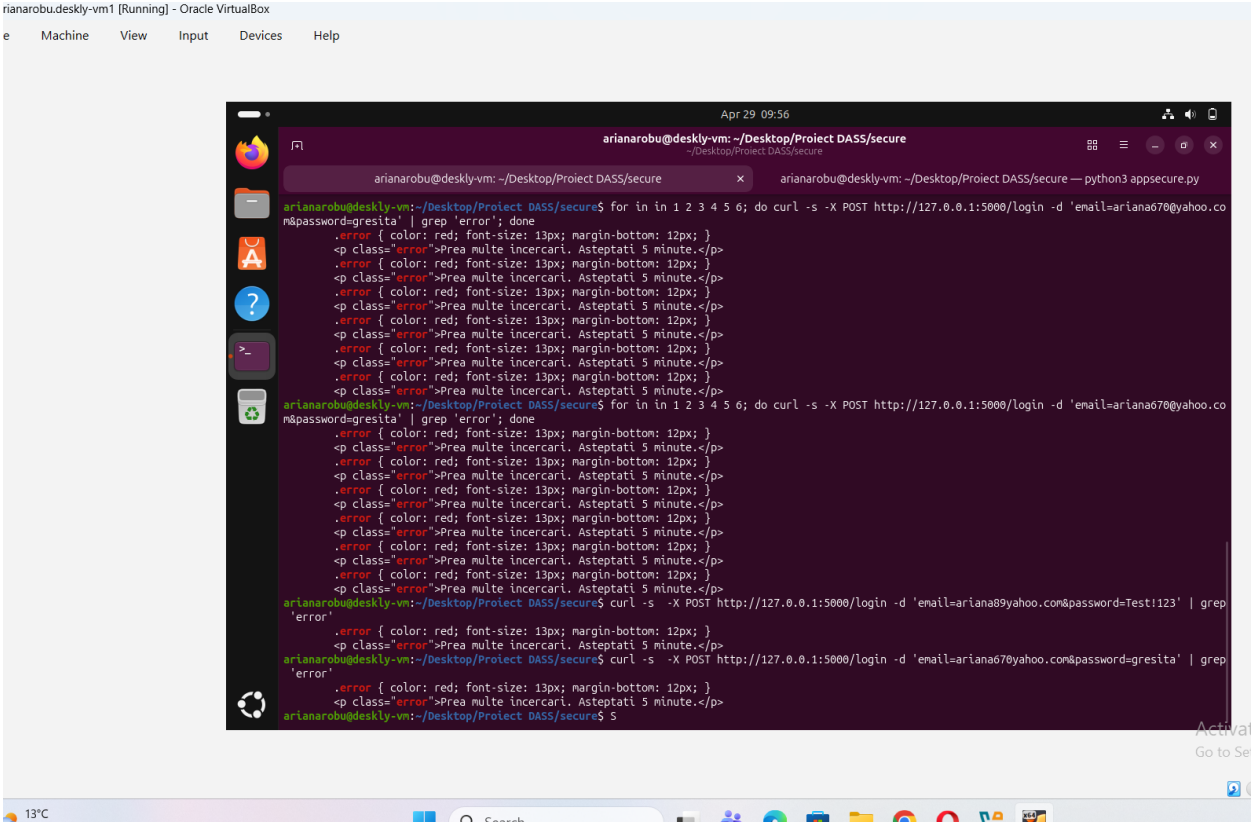
Re-test 4.2.Hash MD5



Rezultat asteptat:

Hash-urile încep cu "$2b$12$" (format bcrypt). Nu exista nicio parola în clar sau hash MD5.

Re-test 4.3.Rate limiting

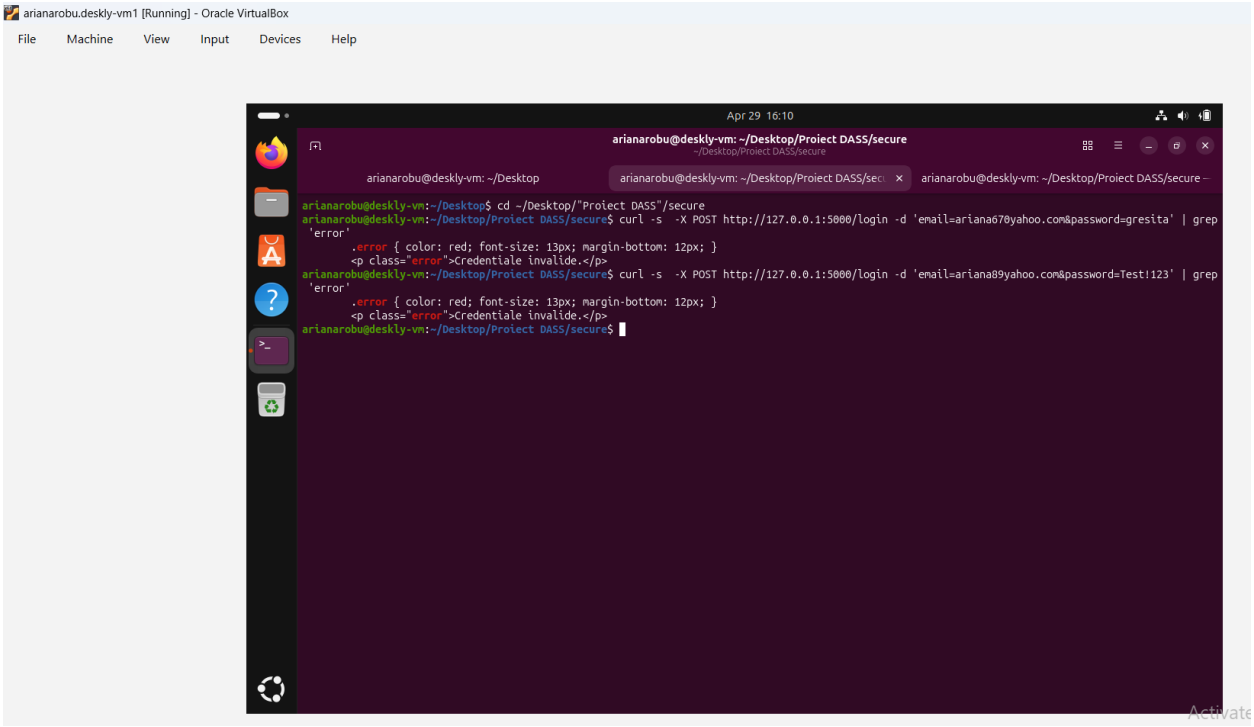


Rezultat asteptat

In versiunea securizata, dupa depasirea limitei de 5 incercari, toate request-urile ulterioare catre /login returneaza eroarea 'Prea multe incercari. Asteptati 5 minute.' IP-ul este blocat automat, atacul de brute force fiind complet neutralizat

Re-test 4.4. User Enumeration

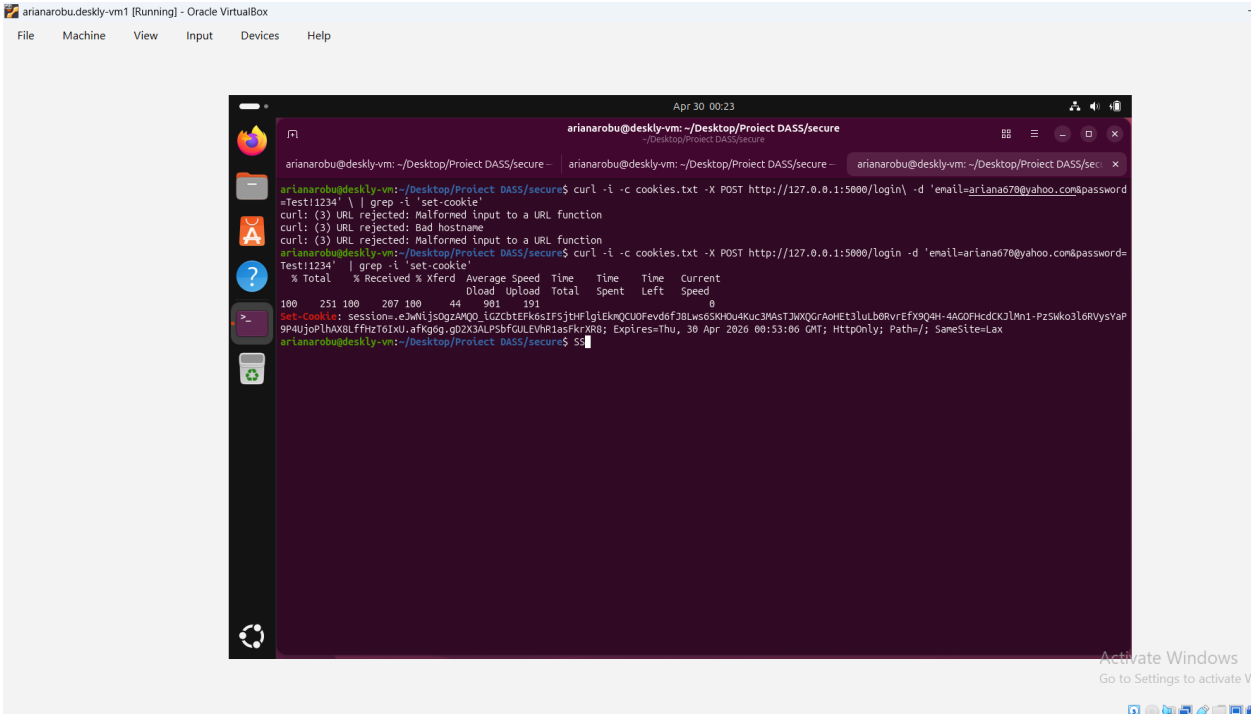
Testam cu email inexistent si cu email existent cu parola gresita:



Rezultat asteptat:

Ambele request-uri returnează acelasi mesaj. Nu mai este posibila diferentierea.

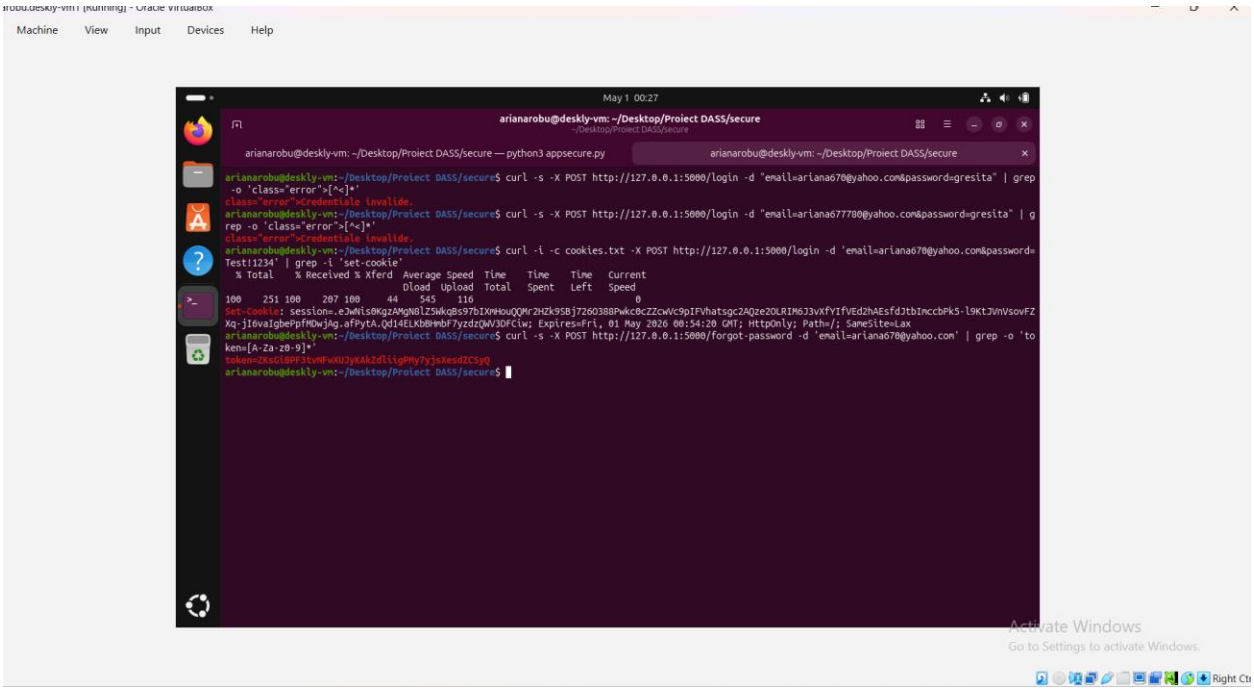
Re-test 4.5.Sesiuni securizate



Rezultat asteptat:

Cookie-ul contine flagurile HttpOnly, SameSite=Lax si Expires — demonstrat in captura de mai sus.

Re-test 4.6.Token resetare



In versiunea securizata, tokenul de resetare este generat cu secrets.token_urlsafe,

rezultand un sir aleator de 43 caractere . Spre deosebire de versiunea vulnerabila unde tokenul era un timestamp predictibil de 10 cifre, acest token nu poate fi ghicit prin forta bruta.

9.Audit&logging

Versiunea securizata implementeaza un sistem complet de audit logging in tabelul audit_logs din baza de date. Fiecare eveniment de securitate relevant este inregistrat cu: user_id, actiunea efectuata, resursa afectata, timestamp si adresa IP.

8.2 – Actiuni inregistrate

LOGIN_SUCCESS — declansata la autentificare reusita — logheaza: user_id, IP, timestamp

LOGIN_FAILED — declansata la credentiale incorecte — logheaza: IP, timestamp (user_id = NULL)

LOGIN_BLOCKED_RATELIMIT — declansata cand IP este blocat de rate limiter — logheaza: IP, timestamp

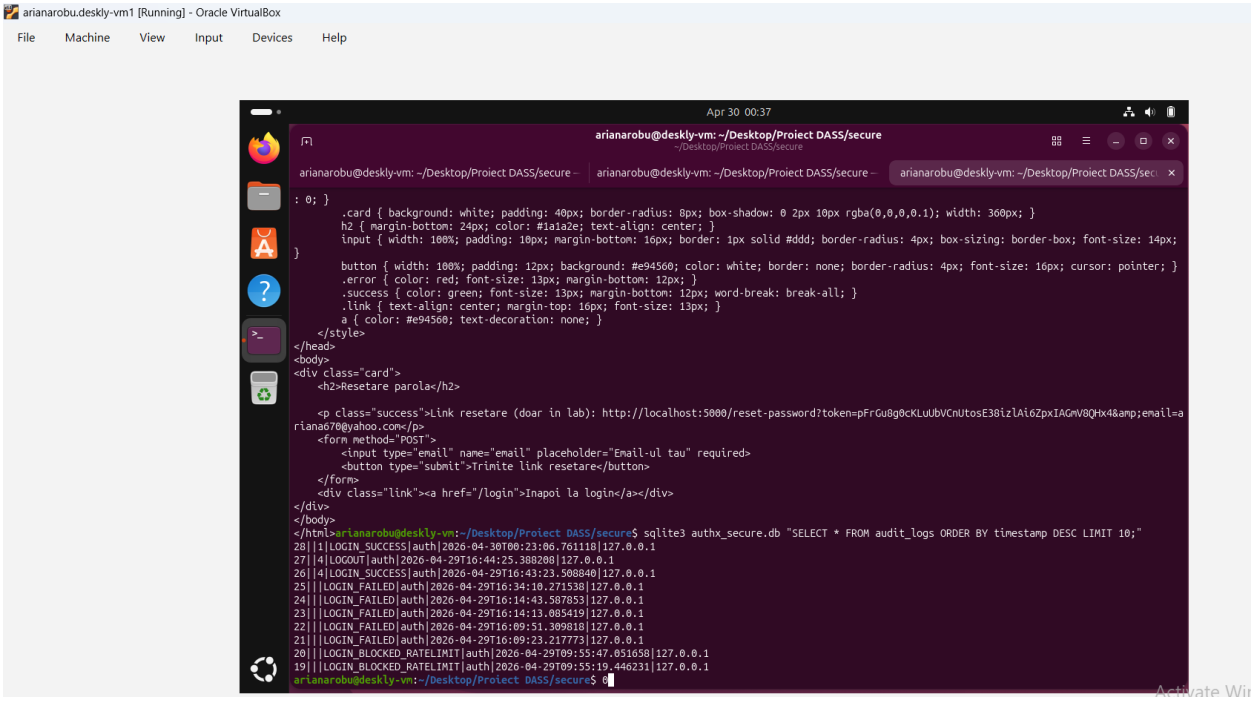
LOGOUT — declansata la apel /logout — logheaza: user_id, IP, timestamp

9.3 – Implementarea functiei de logging

Functia log_action() din appsecure.py centralizeaza toate insertiile in audit_logs.

9.4 – Trasabilitate demonstrata

Prin interogarea tabelului audit_logs, un administrator poate reconstitui complet activitatea unui utilizator sau investiga un incident de securitate.



10.Concluzii

Proiectul "Break the Login" a oferit o perspectiva practica si completa asupra ciclului de viata al securitatii unui sistem de autentificare, de la implementarea deliberat vulnerabila, prin exploatarea vulnerabilitatilor, pana la remedierea si validarea fix-urilor.

10.1 – Lectii invatate

Criptografia conteaza: MD5 si SHA1 nu sunt potrivite pentru stocarea parolelor. bcrypt/argon2 sunt standarde industriale cu un motiv bine justificat — factorul de cost deliberat lent face brute force practic imposibil la scara.

Defense in depth: Nicio singura masura nu este suficienta. Rate limiting + hash puternic + validare parola + mesaje generice formeaza impreuna o aparare eficienta.

Timing attacks sunt reale: Diferenta de timp intre "user inexistent" si "parola gresita" poate fi exploatata. Dummy bcrypt hash-ul din fix-ul 4.4 elimina aceasta diferenta.

Secretele nu stau in cod: app.secret_key = "secret123" hardcodat in repository-ul Git este o vulnerabilitate grava. Variabilele de mediu sunt standard pentru gestionarea secretelor.

Tokenurile trebuie sa fie impredictibile: Un token bazat pe timestamp() poate fi ghicit. secrets.token_urlsafe(32) ofera 256 biti de entropie, complet infezabil de atacat prin forta bruta.

10.2 – Comparatie v1 vs v2

Hash parole — v1: MD5 fara salt — v2: bcrypt rounds=12 cu salt automat

Validare parola — v1: Nicio validare — v2: Min 8 char, upper, digit, special

Rate limiting — v1: Absent — v2: 5 incercari / 5 min / IP

Mesaje eroare — v1: Diferentiate (enumeration) — v2: Generic: "Credentiale invalide."

Cookie sesiune — v1: Fara HttpOnly/Secure/SameSite — v2: HttpOnly + SameSite=Lax + expirare 30min

Secret key — v1: "secret123" hardcodat — v2: secrets.token_hex(32) din ENV

Token reset — v1: Timestamp predictibil, reutilizabil — v2: secrets.token_urlsafe(32), one-time, 15min

Audit logging — v1: Absent — v2: Complet: LOGIN, LOGOUT, BLOCKED

Aplicand principiul "secure by design", versiunea v2 a aplicației AuthX rezista tuturor atacurilor demonstrate în faza de penetration testing, oferind o baza solida pentru un sistem de autentificare de producție.